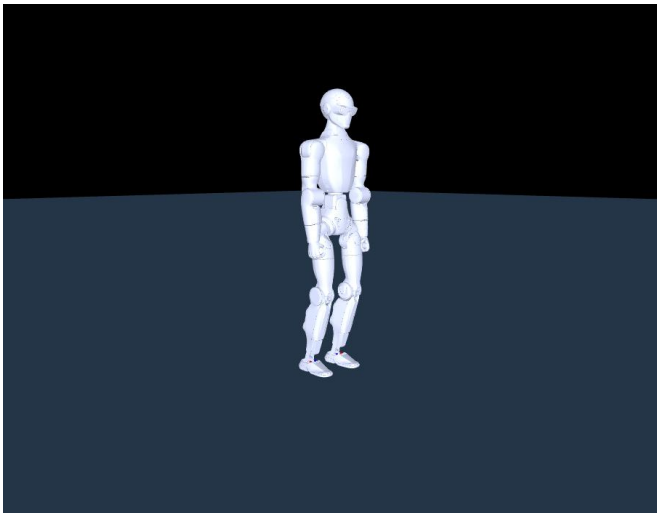
Embodied AI

Lab Manual for MuJoCo

Lab 1 — Make a Humanoid Stand

September 2026



Shaoqin Li

Applied and Embodied AI Lab
Electrical Engineering Department
The City College of New York

you build every file yourself · any laptop · no GPU · start from an empty folder

Contents

Hardware: any laptop, no GPU, no admin rights beyond installing your own software **You start from nothing.** No folder is handed to you. You install the tools, download the robot from the company that makes it, create every file and every directory yourself, and run each one to prove it works before moving on. Nothing below is skippable — a step you skip is a file you do not have.

> **Copy the code from the `.md` version of this manual, not from the PDF.** You were given both. Open `LAB1_make_a_humanoid_stand.md` in VS Code beside this document and copy from there — copying Python out of a PDF drops indentation, and indentation *is* Python.
>
> Every `Expected` block below was produced by actually running the command shown, on a machine built by following these exact steps. **If your numbers differ, that is a finding — tell your instructor.** Verified 2026-09-01 on Ubuntu 24.04 under WSL2, MuJoCo 3.12.0.

What you will do

Part		What you end up with
0	Set up the machine	Linux, conda, MuJoCo, VS Code
1	Build the workspace	four empty directories
2	Get the robot from GitHub	43 mesh files, 22 MB
3	Create the model file	a robot that can be simulated
4	First contact	proof it loads
5	Make it stand	`exp1_pd_sweep.py`, and a standing robot
6	Break it, two completely different ways	the distinction this lab exists for
7	Map all 49 settings	`exp1_pd_sweep.csv`, `exp1_map.png`
8	Push it over, and fail to fix it	the limit of the whole approach
9	See it	pictures and video of your own robot
10	Change the code	`exp1_sandbox.py`, five editable zones
11	Delete it and build it again	the point of the whole lab

What you should be able to say afterwards

1. What `kp` and `kd` do, without using a formula.
2. **How to tell a robot failing from a simulator failing** — and prove which you saw.
3. Why the standing region has a floor *and* a ceiling, from two unrelated causes.
4. One thing this controller can never do, no matter how it is tuned.
5. **Where every file on your disk came from**, and how to make it again from an empty folder.

Part 0 — Set up the machine

Everything in this lab runs on **Linux**. On Windows that means WSL2 — a real Ubuntu inside Windows. Once WSL2 is running, Windows, macOS and Linux users type exactly the same commands for the rest of the lab.

0a — Windows only: install Ubuntu

Open **PowerShell as Administrator** (right-click the Start button → Terminal (Admin)) and run:

```
wsl --install -d Ubuntu-24.04
```

Restart when it asks. Ubuntu opens by itself and asks for a username and password — these are new, for Linux, and are not your Windows login. The password does not echo as you type it. That is normal, not a broken keyboard.

From now on, **every command in this manual is typed in the Ubuntu window**, not in PowerShell. If you closed it, reopen it from the Start menu as "Ubuntu".

0b — macOS and Linux only

Open **Terminal**. That is the whole step. On macOS also run `xcode-select --install` if you have never installed the developer tools — it is what supplies `git`.

0c — Tools

```
sudo apt update && sudo apt install -y git wget
```

macOS: skip this — `git` came from Xcode, and use `curl -O` in place of `wget` in the next step.

Check it landed:

```
git --version
```

Expected:

```
git version 2.43.0
```

Any 2.x is fine.

0d — Install Miniconda

Conda gives you a private Python that cannot break anything else on the machine, and can be deleted by deleting one folder.

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O ~/miniconda.sh
bash ~/miniconda.sh -b -p $HOME/miniconda3
$HOME/miniconda3/bin/conda init bash
```

On an Apple Silicon Mac use `Miniconda3-latest-MacOSX-arm64.sh`; on an Intel Mac, `Miniconda3-latest-MacOSX-x86_64.sh`.

Close the terminal and open a new one. `conda init` edits your shell's startup file, and only a fresh shell reads it. Then:

```
conda --version
```

Expected:

```
conda 26.5.3
```

Your number will differ — a fresh install gives whatever is current. Any recent conda is fine. If instead you get `conda: command not found`, you are still in the old shell. Close it again.

> **Already have conda?** Do not install a second one. Skip straight to 0e — the environment you create there is isolated regardless of which conda made it.

0e — Create the environment

```
conda create -y -n r1lab python=3.11
conda activate r1lab
```

Your prompt now starts with `(r1lab)`. **If it does not, nothing after this point will work.** Every new terminal you open needs `conda activate r1lab` again — this is the single most common reason a command that worked yesterday fails today.

0f — Install the four libraries

```
pip install "mujoco==3.12.0" numpy matplotlib imageio
```

MuJoCo is the physics simulator. The version is pinned so that your numbers and this manual's numbers are comparable; a different version is not wrong, but it makes "my number differs" ambiguous.

0g — Verify

```
python - <<'EOF'
import sys, mujoco, numpy, matplotlib, imageio
print("python    ", sys.version.split()[0])
print("mujoco    ", mujoco.__version__)
print("numpy     ", numpy.__version__)
print("matplotlib", matplotlib.__version__)
print("imageio   ", imageio.__version__)
EOF
```

Expected:

```
python     3.11.15
mujoco     3.12.0
numpy      2.4.6
matplotlib 3.11.1
imageio    2.37.4
```

> ☑ **Checkpoint: five version numbers, no error.** Everything but `mujoco` may be newer than shown.

0h — VS Code

You need an editor, because in Part 3 and after you create real files. Any editor works; these instructions use VS Code.

- **Windows:** install VS Code *on Windows* from code.visualstudio.com, then install the extension called **WSL** (publisher Microsoft). This is what lets a Windows editor open Linux files.
- **macOS / Linux:** install VS Code, then in it press F1 and run *Shell Command: Install 'code' command in PATH*.

You will test it in Part 1.

Part 1 — Build the workspace

Nothing is extracted for you. You make the folders.

```
mkdir -p ~/r1_lab/exp1/model/assets
cd ~/r1_lab/exp1
pwd
```

Expected:

```
/home/YOURNAME/r1_lab/exp1
```

Four directories, and each one has a job:

Directory	Holds
`~/r1_lab`	everything for this lab; delete this one folder and the lab is gone
`~/r1_lab/exp1`	the experiment — every script you write lives here
`~/r1_lab/exp1/model`	the robot description
`~/r1_lab/exp1/model/assets`	the 3-D shapes the description points at

`mkdir -p` makes a whole chain at once and does not complain if part of it already exists. That is why one command made four levels.

Now open this folder in the editor:

```
code .
```

The first time on Windows this installs a small server inside Ubuntu and takes a minute. A VS Code window opens showing an empty `exp1` folder. **Leave it open** — everything from Part 3 on gets created in it.

> If `code` is not found on Windows, VS Code is installed but the **WSL** extension is not. Install it on the Windows side and reopen Ubuntu.

Part 2 — Get the robot from GitHub

The R1's shape is public. Unitree publishes it in the same repository they use for their own simulator, and that is where you are going to get it — not from your instructor.

The whole repository carries ten robots and is about 380 MB. You want one of them. Git can fetch a single folder:

```
cd ~/r1_lab
git clone --depth 1 --filter=blob:none --sparse \
    https://github.com/unitreerobotics/unitree_mujoco.git
cd unitree_mujoco
git sparse-checkout set unitree_robots/r1
```

The `\` at the end of the first line just says *this command continues below*.

Three flags, three savings: `--depth 1` skips the project's history, `--filter=blob:none` skips file *contents* until something asks for them, and `--sparse` starts with no folders checked out at all. The `sparse-checkout set` line is what then asks for exactly one robot.

```
ls unitree_robots/r1
```

Expected:

```
R1_C++.xml  meshes  scene.xml
```

```
ls unitree_robots/r1/meshes | wc -l
du -sh ~/r1_lab/unitree_mujoco
```

Expected:

```
43
32M
```

43 files, 32 MB instead of 380. Those 43 files are the physical shape of the robot: one 3-D model per rigid part, exported from the CAD the machine was built from. `pelvis_link.STL` is the actual pelvis of the actual R1.

Copy them into your workspace:

```
cp unitree_robots/r1/meshes/* ~/r1_lab/exp1/model/assets/
ls ~/r1_lab/exp1/model/assets | wc -l
```

Expected:

```
43
```

> ☑ **Checkpoint: 43. Not 42, not 0.** If this is 0, the `cp` ran from the wrong directory — `cd ~/r1_lab/unitree_mujoco` and try again.

Keep the `unitree_mujoco` folder. You need `R1_C++.xml` in the next part, and deleting it means cloning again.

Part 3 — Create the model file

You now have the robot's *shapes*, and you have Unitree's own description file, `R1_C++.xml`. What you do **not** have is a robot that can be tested, and the gap between those two things is this part.

Open the file you downloaded and look at it:

```
code ~/r1_lab/unitree_mujoco/unitree_robots/r1/R1_C++.xml
```

An MJCF file — MuJoCo's XML format. It is a tree of `<body>` elements, each with a `<joint>` saying how it may move, an `<inertial>` giving its mass, and `<geom>` elements giving its shape. That is a complete description of a machine, and it is missing four things this lab cannot run without:

- **no floor** — nothing to stand on
- **no actuators** — no way to command a joint anywhere
- **collision shapes made of meshes** — accurate, and far too slow to sweep 49 settings with
- **no sensors, and no sites** to attach them to

Create the file

In VS Code, in your `exp1` folder: right-click the `model` folder → **New File** → name it `r1_standalone.xml`. Paste the whole listing below into it and save.

It is long — 378 lines — but it is one paste, and you are not expected to read all of it. The `<body>` blocks in the middle are Unitree's, unchanged. **Copy it from the `.md` version of this manual**, then come back here for the part that matters: what is in your file that was not in theirs.

```
<mujoco model="r1">
  <compiler angle="radian" meshdir="assets"/>
  <default>
    <joint damping="0.05" armature="0.01" frictionloss="0.2"/>
  </default>

  <default>
    <default class="r1">
      <default class="visual">
        <geom type="mesh" density="0" contype="0" conaffinity="0" group="2"/>
      </default>
      <default class="collision">
        <geom type="capsule" priority="1" condim="6" group="3"/>
        <default class="foot_capsule">
          <geom size="0.01"/>
        </default>
      </default>
      <site rgba="1 0 0 1" group="5"/>
    </default>
  </default>

  <asset>
```

```
    <mesh name="pelvis_link" file="pelvis_link.STL"/>
    <mesh name="left_hip_pitch_link" file="left_hip_pitch_link.STL"/>
    <mesh name="left_hip_roll_link" file="left_hip_roll_link.STL"/>
    <mesh name="left_hip_yaw_link" file="left_hip_yaw_link.STL"/>
    <mesh name="left_knee_link" file="left_knee_link.STL"/>
    <mesh name="left_knee_collision" file="left_knee_collision.STL"/>
    <mesh name="left_ankle_A_link" file="left_ankle_A_link.STL"/>
    <mesh name="left_ankle_A_rod_link" file="left_ankle_A_rod_link.STL"/>
    <mesh name="left_ankle_B_link" file="left_ankle_B_link.STL"/>
    <mesh name="left_ankle_B_rod_link" file="left_ankle_B_rod_link.STL"/>
    <mesh name="left_ankle_pitch_link" file="left_ankle_pitch_link.STL"/>
    <mesh name="left_ankle_roll_link" file="left_ankle_roll_link.STL"/>
    <mesh name="left_ankle_constraint_A_link" file="left_ankle_constraint_A_link.STL"/>
    <mesh name="left_ankle_constraint_B_link" file="left_ankle_constraint_B_link.STL"/>
    <mesh name="right_hip_pitch_link" file="right_hip_pitch_link.STL"/>
    <mesh name="right_hip_roll_link" file="right_hip_roll_link.STL"/>
    <mesh name="right_hip_yaw_link" file="right_hip_yaw_link.STL"/>
    <mesh name="right_knee_link" file="right_knee_link.STL"/>
    <mesh name="right_knee_collision" file="right_knee_collision.STL"/>
    <mesh name="right_ankle_A_link" file="right_ankle_A_link.STL"/>
    <mesh name="right_ankle_A_rod_link" file="right_ankle_A_rod_link.STL"/>
    <mesh name="right_ankle_B_link" file="right_ankle_B_link.STL"/>
    <mesh name="right_ankle_B_rod_link" file="right_ankle_B_rod_link.STL"/>
    <mesh name="right_ankle_pitch_link" file="right_ankle_pitch_link.STL"/>
    <mesh name="right_ankle_roll_link" file="right_ankle_roll_link.STL"/>
    <mesh name="right_ankle_constraint_A_link" file="right_ankle_constraint_A_link.STL"/>
    <mesh name="right_ankle_constraint_B_link" file="right_ankle_constraint_B_link.STL"/>
    <mesh name="imu_in_pelvis_link" file="imu_in_pelvis_link.STL"/>
    <mesh name="waist_roll_link" file="waist_roll_link.STL"/>
    <mesh name="waist_yaw_link" file="waist_yaw_link.STL"/>
    <mesh name="left_shoulder_pitch_link" file="left_shoulder_pitch_link.STL"/>
    <mesh name="left_shoulder_roll_link" file="left_shoulder_roll_link.STL"/>
    <mesh name="left_shoulder_yaw_link" file="left_shoulder_yaw_link.STL"/>
    <mesh name="left_elbow_link" file="left_elbow_link.STL"/>
    <mesh name="left_wrist_roll_link" file="left_wrist_roll_link.STL"/>
    <mesh name="right_shoulder_pitch_link" file="right_shoulder_pitch_link.STL"/>
    <mesh name="right_shoulder_roll_link" file="right_shoulder_roll_link.STL"/>
    <mesh name="right_shoulder_yaw_link" file="right_shoulder_yaw_link.STL"/>
    <mesh name="right_elbow_link" file="right_elbow_link.STL"/>
    <mesh name="right_wrist_roll_link" file="right_wrist_roll_link.STL"/>
    <mesh name="head_pitch_link" file="head_pitch_link.STL"/>
    <mesh name="head_yaw_link" file="head_yaw_link.STL"/>
  </asset>


  <worldbody>
    <geom name="floor" type="plane" size="20 20 0.1" rgba="0.2 0.3 0.4 1" friction="1 0.5 0.5"/>
    <body name="pelvis" pos="0 0 0.74" childclass="r1">
      <inertial pos="0.0269881 0.000184581 -0.0704106"
                quat="0.703476 0.707138 -0.0498219 0.0509351" mass="2.25001"
                diaginertia="0.00646721 0.00620017 0.00381749"/>
      <joint name="floating_base_joint" type="free" limited="false" actuatorfrclimited="false"/>
      <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="pelvis_link"/>
      <geom class="visual" pos="0.0762025 2.19914e-05 -0.0884315" quat="1 0 0 0"
```

```
      rgba="0.79216 0.81961 0.93333 1" mesh="imu_in_pelvis_link"/>
<geom name="pelvis_collision" class="collision" type="sphere" size="0.05"
      pos="0.03 0 -0.08"/>
<site name="imu" size="0.01" pos="0 0 0"/>
<body name="left_hip_pitch_link" pos="0.0325 0.0704672 -0.0902351"
      quat="0.976296 -0.21644 0 0">
  <inertial pos="-0.003907 0.0415 -0.043716" quat="0.977607 0.186708 0.0209888 0.0947863"
            mass="0.935482" diaginertia="0.00105051 0.000997373 0.000565117"/>
  <joint name="left_hip_pitch_joint" pos="0 0 0" axis="0 1 0" range="-2.93215 2.54818"/>
  <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="left_hip_pitch_link"/>
  <body name="left_hip_roll_link" pos="0.0248 0.045 -0.053" quat="0.976296 0.21644 0 0">
    <inertial pos="-0.008002 -0.00206 -0.032798"
              quat="0.917982 -0.152984 0.363871 0.0387668" mass="0.207609"
              diaginertia="0.000298715 0.000236156 0.000181129"/>
    <joint name="left_hip_roll_joint" pos="0 0 0" axis="1 0 0" range="-1.0472 1.74533"/>
    <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="left_hip_roll_link"/>
    <geom name="left_hip_collision" class="collision" size="0.03"
          fromto="-0.025 0 0.025 -0.025 0 -0.025"/>
    <body name="left_hip_yaw_link" pos="-0.0194507 -0.0006 -0.0618">
      <inertial pos="-0.010059 -0.004848 -0.093609"
                quat="0.648721 0.0354742 0.0856084 0.755364" mass="1.73967"
                diaginertia="0.0083895 0.00828665 0.00127385"/>
      <joint name="left_hip_yaw_joint" pos="0 0 0" axis="0 0 1" range="-2.7402 2.7402"/>
      <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="left_hip_yaw_link"/>
      <geom name="left_thigh_collision" class="collision" size="0.03"
            fromto="0 0 -0.025 0 0 -0.125"/>
      <body name="left_knee_link" pos="-0.02315 0.01866 -0.159301">
        <inertial pos="-0.00946428 -0.0182147 -0.135746"
                  quat="0.678998 -0.0104211 0.029549 0.733471" mass="2.37477"
                  diaginertia="0.0128449 0.0125305 0.00195841"/>
        <joint name="left_knee_joint" pos="0 0 0" axis="0 1 0" range="-0.174533 2.42601"/>
        <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="left_knee_link"/>
        <geom name="left_shin_collision" class="collision" size="0.035"
              fromto="0.015 -0.02 -0.05 -0.01 -0.02 -0.2"/>
        <geom name="left_linkage_brace_collision" class="collision" size="0.015"
              fromto="-0.01 -0.02 -0.26 -0.015 -0.02 -0.32"/>
        <body name="left_ankle_pitch_link" pos="-0.0168139 -0.0205811 -0.309175">
          <inertial pos="-0.000104 0 -0.00772" quat="0.707107 0 0 0.707107"
                    mass="0.071231" diaginertia="1.2e-05 7e-06 7e-06"/>
          <joint name="left_ankle_pitch_joint" pos="0 0 0" axis="0 1 0"
                 range="-0.87266 0.57596"/>
          <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                mesh="left_ankle_pitch_link"/>
          <body name="left_ankle_roll_link">
            <inertial pos="0.0296647 0 -0.0326975" quat="0 0.737468 0 0.675382"
                      mass="0.481688" diaginertia="0.00126935 0.00121244 0.000230475"/>
            <joint name="left_ankle_roll_joint" pos="0 0 0" axis="1 0 0"
                   range="-0.2618 0.2618"/>
            <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                  mesh="left_ankle_roll_link"/>
            <geom pos="0.021 0.014 -0.0094" quat="1 0 0 0" class="visual" rgba="1 0 0 1"
                  mesh="left_ankle_constraint_A_link"/>
            <geom pos="0.021 -0.014 -0.0094" quat="1 0 0 0" class="visual" rgba="0 0 1 1"
                  mesh="left_ankle_constraint_B_link"/>
```

```
              <geom name="left_foot1_collision" class="foot_capsule"
                    fromto="0.05 -0.025 -0.045 0.1 -0.025 -0.045"/>
              <geom name="left_foot2_collision" class="foot_capsule"
                    fromto="-0.03 -0.02 -0.045 0.115 -0.02 -0.045"/>
              <geom name="left_foot3_collision" class="foot_capsule"
                    fromto="-0.038 -0.01 -0.045 0.12 -0.01 -0.045"/>
              <geom name="left_foot4_collision" class="foot_capsule"
                    fromto="-0.04 0 -0.045 0.123 0 -0.045"/>
              <geom name="left_foot5_collision" class="foot_capsule"
                    fromto="-0.038 0.01 -0.045 0.12 0.01 -0.045"/>
              <geom name="left_foot6_collision" class="foot_capsule"
                    fromto="-0.03 0.02 -0.045 0.115 0.02 -0.045"/>
              <geom name="left_foot7_collision" class="foot_capsule"
                    fromto="0.05 0.025 -0.045 0.1 0.025 -0.045"/>
              <site name="left_foot" rgba="1 0 0 1" pos="0.04 0 -0.055"/>
            </body>
          </body>
        </body>
      </body>
    </body>
    <body name="right_hip_pitch_link" pos="0.0325 -0.0704672 -0.0902351"
          quat="0.976296 0.21644 0 0">
      <inertial pos="-0.003907 -0.0415 -0.043716"
                quat="0.977607 -0.186708 0.0209888 -0.0947863" mass="0.935482"
                diaginertia="0.00105051 0.000997373 0.000565117"/>
      <joint name="right_hip_pitch_joint" pos="0 0 0" axis="0 1 0" range="-2.93215 2.54818"/>
      <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="right_hip_pitch_link"/>
      <body name="right_hip_roll_link" pos="0.0248 -0.045 -0.053" quat="0.976296 -0.21644 0 0">
        <inertial pos="-0.008002 0.00206 -0.032798"
                  quat="0.917982 0.152984 0.363871 -0.0387668" mass="0.207609"
                  diaginertia="0.000298715 0.000236156 0.000181129"/>
        <joint name="right_hip_roll_joint" pos="0 0 0" axis="1 0 0" range="-1.74533 1.0472"/>
        <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="right_hip_roll_link"/>
        <geom name="right_hip_collision" class="collision" size="0.03"
              fromto="-0.025 0 0.025 -0.025 0 -0.025"/>
        <body name="right_hip_yaw_link" pos="-0.0194507 0.0006 -0.0618">
          <inertial pos="-0.010059 0.004848 -0.093609"
                    quat="0.755364 0.0856084 0.0354742 0.648721" mass="1.73967"
                    diaginertia="0.0083895 0.00828665 0.00127385"/>
          <joint name="right_hip_yaw_joint" pos="0 0 0" axis="0 0 1" range="-2.7402 2.7402"/>
          <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="right_hip_yaw_link"/>
          <geom name="right_thigh_collision" class="collision" size="0.03"
                fromto="0 0 -0.025 0 0 -0.125"/>
          <body name="right_knee_link" pos="-0.02315 -0.01866 -0.159301">
            <inertial pos="-0.00946428 0.0182147 -0.135746"
                      quat="0.733471 0.029549 -0.0104211 0.678998" mass="2.37477"
                      diaginertia="0.0128449 0.0125305 0.00195841"/>
            <joint name="right_knee_joint" pos="0 0 0" axis="0 1 0" range="-0.17453 2.42601"/>
            <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="right_knee_link"/>
            <geom name="right_shin_collision" class="collision" size="0.035"
                  fromto="0.015 0.02 -0.05 -0.01 0.02 -0.2"/>
            <geom name="right_linkage_brace_collision" class="collision" size="0.015"
                  fromto="-0.01 0.02 -0.26 -0.015 0.02 -0.32"/>
```

```
            <body name="right_ankle_pitch_link" pos="-0.0168139 0.0205811 -0.309175">
              <inertial pos="-0.000104 0 -0.00772" quat="0.707107 0 0 0.707107"
                        mass="0.071231" diaginertia="1.2e-05 7e-06 7e-06"/>
              <joint name="right_ankle_pitch_joint" pos="0 0 0" axis="0 1 0"
                     range="-0.87266 0.57596"/>
              <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                    mesh="right_ankle_pitch_link"/>
              <body name="right_ankle_roll_link">
                <inertial pos="0.0296647 0 -0.0326975" quat="0 0.737468 0 0.675382"
                          mass="0.481688" diaginertia="0.00126935 0.00121244 0.000230475"/>
                <joint name="right_ankle_roll_joint" pos="0 0 0" axis="1 0 0"
                       range="-0.261799 0.261799"/>
                <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                      mesh="right_ankle_roll_link"/>
                <geom pos="0.021 -0.014 -0.0094" quat="1 0 0 0" class="visual" rgba="1 0 0 1"
                      mesh="right_ankle_constraint_A_link"/>
                <geom pos="0.021 0.014 -0.0094" quat="1 0 0 0" class="visual" rgba="0 0 1 1"
                      mesh="right_ankle_constraint_B_link"/>
                <geom name="right_foot1_collision" class="foot_capsule"
                      fromto="0.05 -0.025 -0.045 0.1 -0.025 -0.045"/>
                <geom name="right_foot2_collision" class="foot_capsule"
                      fromto="-0.03 -0.02 -0.045 0.115 -0.02 -0.045"/>
                <geom name="right_foot3_collision" class="foot_capsule"
                      fromto="-0.038 -0.01 -0.045 0.12 -0.01 -0.045"/>
                <geom name="right_foot4_collision" class="foot_capsule"
                      fromto="-0.04 0 -0.045 0.123 0 -0.045"/>
                <geom name="right_foot5_collision" class="foot_capsule"
                      fromto="-0.038 0.01 -0.045 0.12 0.01 -0.045"/>
                <geom name="right_foot6_collision" class="foot_capsule"
                      fromto="-0.03 0.02 -0.045 0.115 0.02 -0.045"/>
                <geom name="right_foot7_collision" class="foot_capsule"
                      fromto="0.05 0.025 -0.045 0.1 0.025 -0.045"/>
                <site name="right_foot" rgba="1 0 0 1" pos="0.04 0 -0.055"/>
              </body>
            </body>
          </body>
        </body>
      </body>
      <body name="waist_roll_link">
        <inertial pos="0.030601 0 0.004788" quat="0.999992 0 -0.00390592 0" mass="0.920736"
                  diaginertia="0.000806008 0.000734 0.000677992"/>
        <joint name="waist_roll_joint" pos="0 0 0" axis="1 0 0" range="-0.5236 0.5236"/>
        <geom pos="0.0325 0 0.049" class="visual" rgba="0.79216 0.81961 0.93333 1"
              mesh="waist_roll_link"/>
        <body name="torso_link" pos="0.0325 0 0.049">
          <inertial pos="0.000409289 -0.000333222 0.181216"
                    quat="0.999992 -0.000325591 0.00182329 -0.00361781" mass="7.3387"
                    diaginertia="0.110151 0.10028 0.0271256"/>
          <joint name="waist_yaw_joint" pos="0 0 0" axis="0 0 1" range="-2.618 2.618"/>
          <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="waist_yaw_link"/>
          <geom pos="-0.006 0.03155 0.255" class="visual" rgba="0.79216 0.81961 0.93333 1"
                mesh="head_pitch_link"/>
          <geom pos="-0.0049874 0 0.3715" quat="1 0 0 0" class="visual"
```

```
      rgba="0.79216 0.81961 0.93333 1" mesh="head_yaw_link"/>
<geom name="torso_collision" class="collision" size="0.08"
      fromto="-0.02 0 0.1 -0.02 0 0.15"/>
<geom name="head_collision" class="collision" type="sphere" size="0.055"
      pos="0.015 0 0.35"/>
<body name="left_shoulder_pitch_link" pos="0 0.085688 0.19749"
      quat="0.99144 0.130561 0 0">
  <inertial pos="0.00294386 0.0447687 -0.0228687"
            quat="0.933514 0.315801 -0.107611 0.131308" mass="0.83795"
            diaginertia="0.000664554 0.000644431 0.000530845"/>
  <joint name="left_shoulder_pitch_joint" pos="0 0 0" axis="0 1 0"
         range="-3.1416 2.0944"/>
  <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
        mesh="left_shoulder_pitch_link"/>
  <body name="left_shoulder_roll_link" pos="0.03445 0.047132 -0.025693"
        quat="0.99144 -0.130561 0 0">
    <inertial pos="-0.037828 0.003967 -0.066496"
              quat="0.704928 -0.0676661 -0.0976966 0.699252" mass="0.740212"
              diaginertia="0.00108994 0.000995148 0.000547915"/>
    <joint name="left_shoulder_roll_joint" pos="0 0 0" axis="1 0 0"
           range="-0.22689 2.4784"/>
    <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
          mesh="left_shoulder_roll_link"/>
    <body name="left_shoulder_yaw_link" pos="-0.03445 0.0043 -0.10835">
      <inertial pos="0.013143 -0.004355 -0.072948"
                quat="0.944453 -0.083861 -0.114993 0.29623" mass="0.738441"
                diaginertia="0.000862551 0.000851784 0.000435665"/>
      <joint name="left_shoulder_yaw_joint" pos="0 0 0" axis="0 0 1"
             range="-1.9199 1.9199"/>
      <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
            mesh="left_shoulder_yaw_link"/>
      <geom name="left_shoulder_yaw_collision" class="collision" size="0.03"
            fromto="0 0 -0.03 0 0 0.1"/>
      <body name="left_elbow_link" pos="0.016191 0.026461 -0.082858">
        <inertial pos="0.072371 -0.024612 -0.011566"
                  quat="0.590432 0.585258 0.4407 0.338595" mass="0.759419"
                  diaginertia="0.000969031 0.000938046 0.000470923"/>
        <joint name="left_elbow_joint" pos="0 0 0" axis="0 1 0"
               range="-0.97564 2.1852"/>
        <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="left_elbow_link"/>
        <geom name="left_elbow_collision" class="collision" size="0.03"
              fromto="0.015 -0.025 -0.01 0.085 -0.025 -0.01"/>
        <body name="left_wrist_roll_link" pos="0.11218 -0.03002 -0.011702">
          <inertial pos="0.068045 -0.001688 0.001392"
                    quat="0.442664 0.566917 0.411512 0.559742" mass="0.324767"
                    diaginertia="0.000717953 0.000704181 0.000156865"/>
          <joint name="left_wrist_roll_joint" pos="0 0 0" axis="1 0 0"
                 range="-1.9199 1.9199"/>
          <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                mesh="left_wrist_roll_link"/>
          <geom name="left_hand_collision" class="collision" size="0.03"
                rgba="0.2 0.6 0.2 0.2" fromto="0.05 0 0 0.12 0 0"/>
          <site name="left_palm" pos="0.1 0 0" size="0.01"/>
        </body>
```

```
      </body>
    </body>
  </body>
</body>
<body name="right_shoulder_pitch_link" pos="0 -0.085688 0.19749"
      quat="0.99144 -0.130561 0 0">
  <inertial pos="0.00294386 -0.0447687 -0.0228687"
            quat="0.933514 -0.315801 -0.107611 -0.131308" mass="0.83795"
            diaginertia="0.000664554 0.000644431 0.000530845"/>
  <joint name="right_shoulder_pitch_joint" pos="0 0 0" axis="0 1 0"
         range="-3.1416 2.0944"/>
  <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
        mesh="right_shoulder_pitch_link"/>
  <body name="right_shoulder_roll_link" pos="0.03445 -0.047132 -0.025693"
        quat="0.99144 0.130561 0 0">
    <inertial pos="-0.037828 -0.003967 -0.066496"
              quat="0.699252 -0.0976966 -0.0676661 0.704928" mass="0.740212"
              diaginertia="0.00108994 0.000995148 0.000547915"/>
    <joint name="right_shoulder_roll_joint" pos="0 0 0" axis="1 0 0"
           range="-2.47849 0.2268"/>
    <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
          mesh="right_shoulder_roll_link"/>
    <body name="right_shoulder_yaw_link" pos="-0.03445 -0.0043 -0.10835">
      <inertial pos="0.013143 0.004355 -0.072948"
                quat="0.944453 0.083861 -0.114993 -0.29623" mass="0.738441"
                diaginertia="0.000862551 0.000851784 0.000435665"/>
      <joint name="right_shoulder_yaw_joint" pos="0 0 0" axis="0 0 1"
             range="-1.9199 1.9199"/>
      <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
            mesh="right_shoulder_yaw_link"/>
      <geom name="right_shoulder_yaw_collision" class="collision" size="0.03"
            fromto="0 0 -0.03 0 0 0.1"/>
      <body name="right_elbow_link" pos="0.016191 -0.026461 -0.082858">
        <inertial pos="0.072371 0.024612 -0.011566"
                  quat="0.338595 0.4407 0.585258 0.590432" mass="0.759419"
                  diaginertia="0.000969031 0.000938046 0.000470923"/>
        <joint name="right_elbow_joint" pos="0 0 0" axis="0 1 0"
               range="-0.97564 2.1852"/>
        <geom class="visual" rgba="0.79216 0.81961 0.93333 1" mesh="right_elbow_link"/>
        <geom name="right_elbow_collision" class="collision" size="0.03"
              fromto="0.015 0.025 -0.01 0.085 0.025 -0.01"/>
        <body name="right_wrist_roll_link" pos="0.11218 0.03002 -0.011702">
          <inertial pos="0.068045 0.001688 0.001392"
                    quat="0.559742 0.411512 0.566917 0.442664" mass="0.324767"
                    diaginertia="0.000717953 0.000704181 0.000156865"/>
          <joint name="right_wrist_roll_joint" pos="0 0 0" axis="1 0 0"
                 range="-1.9199 1.9199"/>
          <geom class="visual" rgba="0.79216 0.81961 0.93333 1"
                mesh="right_wrist_roll_link"/>
          <geom name="right_hand_collision" class="collision" size="0.03"
                rgba="0.2 0.6 0.2 0.2" fromto="0.05 0 0 0.12 0 0"/>
          <site name="right_palm" pos="0.1 0 0" size="0.01"/>
        </body>
      </body>
```

```
            </body>
          </body>
        </body>
      </body>
    </body>
  </body>
 </worldbody>
 <contact>
   <exclude body1="pelvis" body2="right_hip_roll_link"/>
   <exclude body1="pelvis" body2="left_hip_roll_link"/>
 </contact>
 <sensor>
   <gyro name="imu_ang_vel" site="imu"/>
   <velocimeter name="imu_lin_vel" site="imu"/>
   <accelerometer name="imu_lin_acc" site="imu"/>
   <subtreeangmom name="root_angmom" body="pelvis"/>
 </sensor>
 <actuator>
   <position name="left_hip_pitch"    joint="left_hip_pitch_joint"    kp="600" dampratio="1"/>
   <position name="left_hip_roll"     joint="left_hip_roll_joint"     kp="600" dampratio="1"/>
   <position name="left_hip_yaw"      joint="left_hip_yaw_joint"      kp="600" dampratio="1"/>
   <position name="left_knee"         joint="left_knee_joint"         kp="600" dampratio="1"/>
   <position name="left_ankle_pitch"  joint="left_ankle_pitch_joint"  kp="300" dampratio="1"/>
   <position name="left_ankle_roll"   joint="left_ankle_roll_joint"   kp="300" dampratio="1"/>
   <position name="right_hip_pitch"   joint="right_hip_pitch_joint"   kp="600" dampratio="1"/>
   <position name="right_hip_roll"    joint="right_hip_roll_joint"    kp="600" dampratio="1"/>
   <position name="right_hip_yaw"     joint="right_hip_yaw_joint"     kp="600" dampratio="1"/>
   <position name="right_knee"        joint="right_knee_joint"        kp="600" dampratio="1"/>
   <position name="right_ankle_pitch" joint="right_ankle_pitch_joint" kp="300" dampratio="1"/>
   <position name="right_ankle_roll"  joint="right_ankle_roll_joint"  kp="300" dampratio="1"/>
 </actuator>
</mujoco>
```

What you actually changed

You did not type that from nothing — it is Unitree's file with edits. Look at the edits themselves:

```
cd ~/r1_lab/exp1
wc -l ~/r1_lab/unitree_mujoco/unitree_robots/r1/R1_C++.xml model/r1_standalone.xml
```

Expected:

```
  381 /home/YOURNAME/r1_lab/unitree_mujoco/unitree_robots/r1/R1_C++.xml
  378 model/r1_standalone.xml
  759 total
```

```
U=~/r1_lab/unitree_mujoco/unitree_robots/r1/R1_C++.xml
diff $U model/r1_standalone.xml | grep -c "^<"
diff $U model/r1_standalone.xml | grep -c "^>"
```

Expected:

```
283
280
```

283 lines removed, 280 added — almost nothing survived untouched. Nine changes, and every one of them is a decision someone had to make:

#	Change	Why
1	`meshdir` `meshes/` → `assets`	your folder is called `assets`; the model must be told where its shapes are
2	joint `damping` 0.001 → 0.05, `frictionloss` 0.1 → 0.2	real joints have grease and gearing; the shipped values model a nearly frictionless ideal
3	a `<default class="r1">` block with `visual`, `collision`, `foot_capsule` children	so a hundred geoms can be re-styled by editing three lines
4	`actuatorfrcrange` deleted from every joint	Part 6 needs to see torque go where the physics sends it, not get clipped at a limit
5	**collision meshes replaced by capsules** — pelvis sphere, hip, thigh, shin, torso, head, hands	the single biggest change: contact against a 3-D mesh is exact and slow, contact against a capsule is approximate and fast enough to run 49 trials
6	**seven thin capsules per foot** instead of one box	a box tips on an edge; seven capsules in a row give the foot a believable rolling contact patch
7	**a floor** — `<geom name="floor" type="plane" size="20 20 0.1" friction="1 0.5 0.5"/>`	Unitree's file has none. Their simulator adds it in `scene.xml`. Yours adds it directly
8	`<site>` markers and an IMU `<sensor>` block	sensors have to be bolted to somewhere specific
9	**twelve `<position>` actuators, `kp="600"` / `kp="300"`, `dampratio="1"`**	**this is the lab.** The two dials you spend the next two hours turning do not exist in the file you downloaded

> Point 9 is worth sitting with. The subject of this entire experiment — the stiffness and damping of the leg joints — is not a property of the robot Unitree ships. It is something *you* added. A physical robot has a shape and a mass; how hard it holds a pose is a choice made by whoever writes the controller.

Part 4 — First contact

Before any physics, prove the model loads and finds its 43 shapes. In VS Code, in `exp1`, create `check_model.py`:

```
"""First contact: load the model, print what MuJoCo found, and stop.

No physics, no window.  If this prints numbers, the model file and all 43
meshes are where they need to be and the install works.
"""
import mujoco

model = mujoco.MjModel.from_xml_path("model/r1_standalone.xml")

print("loaded model/r1_standalone.xml")
print("  bodies            ", model.nbody)
print("  joints            ", model.njnt)
print("  position slots    ", model.nq)
print("  actuators         ", model.nu)
print("  meshes            ", model.nmesh)
print("  total mass  %.2f kg" % sum(model.body_mass))
print("  timestep    %.4f s" % model.opt.timestep)
```

Run it:

```
python check_model.py
```

Expected:

```
loaded model/r1_standalone.xml
  bodies             26
  joints             25
  position slots     31
  actuators          12
  meshes             42
  total mass  28.93 kg
  timestep    0.0020 s
```

Read four of those:

- **12 actuators** — six joints per leg, and nothing else on the robot is driven. The arms and head are along for the ride.
- **25 joints but 31 position slots.** The extra six are the pelvis floating in space: three for where it is, and four for how it is rotated (a quaternion), which is 7 numbers for 1 "joint".
- **28.93 kg.** Remember this in Part 6 — it is the weight the ankles have to hold up.

- **42 meshes, and you copied 43 files.** One is unused: `torso_collision.stl`, which the file you wrote replaced with a capsule in change #5. Nothing is wrong. It is evidence you really did swap the collision geometry.

> ☑ **Checkpoint: this must print numbers before you go on.** A `resource not found` error naming a `.STL` file means Part 2's copy did not land — recount `ls model/assets | wc -l`.

Part 5 — Make it stand

Now the experiment itself. In VS Code, in `exp1`, create **`exp1_pd_sweep.py`** and paste this in. It is the longest file in the lab and the only one that does physics; everything after it either draws its output or lets you edit its ideas.

```
"""Workshop experiment 1 -- can the R1 stand with no learning at all?

Pure PD position control, no policy, no training.  Sweeps the two gains a
student actually turns and records whether the robot stands, collapses, or
shakes itself apart.

MuJoCo's <position> actuator is already a PD law:

    tau = kp * (target - q) - kv * qdot

and after compilation those two gains live in gainprm[0] and -biasprm[2], so we
can retune them at runtime without touching the XML.  That is the whole point of
the experiment: one number controls how hard the joint pulls toward its target,
the other controls how hard it resists moving, and standing lives in a bounded
region of that plane.

Usage:
    python exp1_pd_sweep.py                 # full sweep -> csv
    python exp1_pd_sweep.py --kp 600 --kd 30 --seconds 10   # single trial
"""
import argparse
import csv
import math
import os

import mujoco
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))

# Where the robot model lives.  This used to be a hardcoded absolute path under
# /home/sql, which meant the script ran on exactly one machine.  Resolution order:
#   1. --xml on the command line
#   2. $R1_XML
#   3. a model shipped next to this script
#   4. the unitree_rl_mjlab checkout, wherever the user cloned it
```

```
# The model needs its meshes/ directory alongside it, so we resolve the XML and
# let MuJoCo pick up assets via the XML's own relative meshdir.
XML_CANDIDATES = [
    os.path.join(HERE, "model", "r1_standalone.xml"),
    os.path.join(HERE, "r1_standalone.xml"),
    os.path.expanduser("~/unitree_rl_mjlab/src/assets/robots/unitree_r1/xmls/r1_standalone.xml"),
]


def resolve_xml(explicit=None):
    """Return the first candidate that actually LOADS.

    Existence is not enough: r1_standing/ contains an orphaned copy of the XML
    whose assets/ mesh directory was never copied alongside it, so it passes an
    os.path.exists check and then dies inside MuJoCo complaining about
    pelvis_link.STL.  Trying the load is the only honest test.
    """
    tried = []
    for cand in ([explicit] if explicit else []) + [os.environ.get("R1_XML")] + XML_CANDIDATES:
        if not cand or not os.path.exists(cand):
            continue
        try:
            mujoco.MjModel.from_xml_path(cand)
            return cand
        except Exception as e:
            tried.append(f"{cand}\n      -> {str(e).splitlines()[0][:90]}")
    if tried:
        raise SystemExit("Found model file(s), but none could be loaded:\n   " +
                         "\n   ".join(tried) +
                         "\n\nThe XML needs its assets/ mesh folder beside it.")
    raise SystemExit(
        "Could not find the R1 model.\n"
        "Looked for:\n  " + "\n  ".join(c for c in XML_CANDIDATES if c) + "\n"
        "Fix by either:\n"
        "  export R1_XML=/path/to/r1_standalone.xml\n"
        "  python3 exp1_pd_sweep.py --xml /path/to/r1_standalone.xml")


XML_PATH = None   # set in main() / build()

# A nominal crouched standing pose: slight hip pitch, bent knee, matching ankle.
STAND_POS = {
    "left_hip_pitch": -0.1, "left_hip_roll": 0.0, "left_hip_yaw": 0.0,
    "left_knee": 0.3, "left_ankle_pitch": -0.2, "left_ankle_roll": 0.0,
    "right_hip_pitch": -0.1, "right_hip_roll": 0.0, "right_hip_yaw": 0.0,
    "right_knee": 0.3, "right_ankle_pitch": -0.2, "right_ankle_roll": 0.0,
}


def build():
    model = mujoco.MjModel.from_xml_path(XML_PATH or resolve_xml())
    data = mujoco.MjData(model)
    act = {mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_ACTUATOR, i): i
           for i in range(model.nu)}
```

```
    jnt = {mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_JOINT, i): i
           for i in range(model.njnt)}
    return model, data, act, jnt


def set_gains(model, act, kp, kd):
    """Override every actuator's PD gains in place."""
    for i in act.values():
        model.actuator_gainprm[i, 0] = kp
        model.actuator_biasprm[i, 1] = -kp
        model.actuator_biasprm[i, 2] = -kd


def reset_to_stand(model, data, act, jnt):
    mujoco.mj_resetData(model, data)
    for name, angle in STAND_POS.items():
        jname = name + "_joint"
        if jname in jnt:
            data.qpos[model.jnt_qposadr[jnt[jname]]] = angle
        if name in act:
            data.ctrl[act[name]] = angle
    mujoco.mj_forward(model, data)


def tilt_deg(data):
    """Angle between the pelvis z-axis and world up."""
    zaxis = data.xmat[1].reshape(3, 3)[:, 2]
    return math.degrees(math.acos(np.clip(zaxis[2], -1.0, 1.0)))


def trial(kp, kd, seconds=10.0, fall_frac=0.6, perturb=0.0):
    """Run one standing trial.  Returns a dict of outcome metrics."""
    model, data, act, jnt = build()
    set_gains(model, act, kp, kd)
    reset_to_stand(model, data, act, jnt)

    h0 = data.xpos[1][2]                      # pelvis height at t=0
    steps = int(seconds / model.opt.timestep)
    fall_t = None
    max_tilt = 0.0
    late_qvel, late_err, torques = [], [], []
    targets = np.array([STAND_POS[mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_ACTUATOR, i)]
                        for i in range(model.nu)])

    for s in range(steps):
        if perturb and s == int(1.0 / model.opt.timestep):
            data.qvel[0] += perturb           # sideways shove after 1 s

        mujoco.mj_step(model, data)

        # Too much damping makes the explicit integrator diverge -- a distinct
        # failure from falling over.  Checking qpos for NaN is NOT enough:
        # MuJoCo detects the blow-up itself and silently resets the state, so
        # the trajectory looks plausible again a few steps later while |qvel|
```

```
        # has passed 1e6 in between.  The warning counter is the honest signal.
        diverged = (data.warning[mujoco.mjtWarning.mjWARN_BADQACC].number > 0
                    or np.abs(data.qvel).max() > 100.0
                    or np.abs(data.actuator_force).max() > 1e4)
        if diverged or not (np.isfinite(data.qpos).all() and np.isfinite(data.qvel).all()):
            # If the robot had already toppled, the blow-up is a *consequence*
            # of the collapse (limbs slamming into limits), not a gain problem.
            # Report the physical failure -- it happened first and it is the
            # one the student is meant to see.
            return {"kp": kp, "kd": kd, "stood": False,
                    "fall_t": round(fall_t if fall_t is not None else s * model.opt.timestep, 3),
                    "height_drop_cm": float("nan"), "max_tilt_deg": float("nan"),
                    "osc_qvel_rms": float("nan"), "track_err_rad": float("nan"),
                    "peak_torque_Nm": float("nan"), "diverged": fall_t is None}

        h = data.xpos[1][2]
        max_tilt = max(max_tilt, tilt_deg(data))

        if fall_t is None and (h < fall_frac * h0 or max_tilt > 45.0):
            fall_t = s * model.opt.timestep

        if s > steps - int(1.0 / model.opt.timestep):   # last second
            qv = np.array([data.qvel[model.jnt_dofadr[jnt[
                mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_ACTUATOR, i) + "_joint"]]]
                for i in range(model.nu)])
            qp = np.array([data.qpos[model.jnt_qposadr[jnt[
                mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_ACTUATOR, i) + "_joint"]]]
                for i in range(model.nu)])
            late_qvel.append(np.sqrt(np.mean(qv ** 2)))
            late_err.append(np.max(np.abs(qp - targets)))
        torques.append(np.max(np.abs(data.actuator_force)))

    return {
        "kp": kp, "kd": kd,
        "stood": fall_t is None,
        "fall_t": round(fall_t, 3) if fall_t is not None else seconds,
        "height_drop_cm": round(100 * (h0 - data.xpos[1][2]), 2),
        "max_tilt_deg": round(max_tilt, 2),
        "osc_qvel_rms": round(float(np.mean(late_qvel)) if late_qvel else float("nan"), 4),
        "track_err_rad": round(float(np.mean(late_err)) if late_err else float("nan"), 4),
        "peak_torque_Nm": round(float(np.max(torques)), 1),
        "diverged": False,
    }


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--xml", default=None, help="path to r1_standalone.xml")
    ap.add_argument("--kp", type=float, default=None)
    ap.add_argument("--kd", type=float, default=None)
    ap.add_argument("--seconds", type=float, default=10.0)
    ap.add_argument("--perturb", type=float, default=0.0,
                    help="sideways base velocity impulse at t=1s (m/s)")
    ap.add_argument("--out", default=os.path.join(HERE, "exp1_pd_sweep.csv"))
```

```
    args = ap.parse_args()
    global XML_PATH
    XML_PATH = resolve_xml(args.xml)
    print(f"model: {XML_PATH}")

    if args.kp is not None and args.kd is not None:
        r = trial(args.kp, args.kd, args.seconds, perturb=args.perturb)
        for k, v in r.items():
            print(f"  {k:16s} {v}")
        return

    kps = [25, 50, 100, 200, 400, 600, 900]
    kds = [0, 1, 5, 15, 30, 60, 120]
    rows = []
    print(f"sweeping {len(kps)}x{len(kds)} = {len(kps)*len(kds)} trials "
          f"of {args.seconds}s each\n")
    header = "kp\\kd " + "".join(f"{kd:>8}" for kd in kds)
    print(header)
    for kp in kps:
        line = f"{kp:>5} "
        for kd in kds:
            r = trial(kp, kd, args.seconds, perturb=args.perturb)
            rows.append(r)
            if r["stood"]:
                # distinguish a clean stand from a shaky one
                mark = "OK" if r["osc_qvel_rms"] < 0.05 else "buzz"
            elif r["diverged"]:
                mark = "DIVERGE"
            else:
                mark = f"fall{r['fall_t']:.1f}"
            line += f"{mark:>8}"
        print(line)

    with open(args.out, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        w.writeheader()
        w.writerows(rows)
    print(f"\nwrote {args.out}")


if __name__ == "__main__":
    main()
```

Run one setting:

```
python exp1_pd_sweep.py --kp 600 --kd 30 --seconds 5
```

Expected: *(the script first prints the path it resolved; that line is omitted here and in every block below, because it contains the username of the machine it ran on)*

```
kp               600.0
kd               30.0
stood            True
fall_t           5.0
height_drop_cm   0.77
max_tilt_deg     1.15
osc_qvel_rms     0.0
track_err_rad    0.0174
peak_torque_Nm   31.4
diverged         False
```

You just balanced a 12-joint humanoid on a machine you set up an hour ago. **There is no neural network in there** — no training, no AI. The rule underneath is one sentence, applied 500 times a second to each leg joint: *if this joint is not where I want it, push it back.*

Field	Meaning
`stood`	survived the trial
`fall_t`	when it fell; equals trial length if it never did
`height_drop_cm`	how far the hips sank
`max_tilt_deg`	worst lean, in degrees
`osc_qvel_rms`	leftover jitter — 0.0 means genuinely still
`track_err_rad`	**worst joint's distance from its target**
`peak_torque_Nm`	hardest any joint pushed
`diverged`	did the simulator itself fail

Stop on `track_err_rad = 0.0174`. That is about 1°. The robot is standing perfectly and *still* sits a degree away from where it was told to be — permanently. That is not a bug. It is the entire explanation for Part 7's floor.

> ☑ **Checkpoint: you need `stood True` before continuing.**

Part 6 — Break it, two ways

There are two failure modes here. Confusing them is the most common mistake in this lab, and telling them apart is the most useful thing you will take away.

6a — Weak spring

```
python exp1_pd_sweep.py --kp 100 --kd 5 --seconds 5
```

Expected:

```
stood            False
fall_t           1.142
height_drop_cm   65.03
max_tilt_deg     90.93
track_err_rad    0.0497
peak_torque_Nm   17.4
diverged         False
```

Fell after 1.14 s, dropped 65 cm, tilted 91°. Three things, in order of importance:

1. `diverged` **is** `False` **and every value is a real number.** The physics worked. A real robot would do exactly this.
2. `track_err_rad` **went 0.0174 → 0.0497.** You cut `kp` sixfold and the permanent lean roughly tripled. That lean is what killed it.
3. `peak_torque_Nm` **is 17.4 —** ***lower*** **than the successful run's 31.4.**

> Read that third point twice. The robot that **fell** never pushed as hard as the one that **stood**. It had 31 N·m available and used 17. **It did not fall from weakness. It fell because it was leaning.**

6b — Too much damping

```
python exp1_pd_sweep.py --kp 600 --kd 120 --seconds 5
```

Expected:

```
stood            False
fall_t           0.044
height_drop_cm   nan
max_tilt_deg     nan
track_err_rad    nan
peak_torque_Nm   nan
diverged         True
```

`diverged True`, everything `nan`, "failed" after 22 simulator steps. **Nothing physical happened.** The arithmetic ran away to infinity. A real robot cannot do this — you are watching your *tool* fail.

The distinction

	6a — toppled	6b — blew up
`diverged`	`False`	`True`
measurements	real numbers	`nan`

	6a — toppled	6b — blew up
`fall_t`	tenths of a second to seconds	usually under 0.1 s
would a real robot do this?	**yes**	**no**

> **Why not just check for NaN?** When the solver explodes, MuJoCo detects it and *silently resets the state*. A few steps later the trajectory looks plausible again while velocities passed 10^6 in between. Sample a few frames and it all looks fine. The script reads MuJoCo's own warning counter instead — you pasted that in Part 5, it is the `warning[mjWARN_BADQACC]` line. That is the honest signal.

> ☑ **Checkpoint: produce both, and point at the field that separates them.**

Part 7 — Map all 49 settings

Predict first

Seriously. The reveal is worthless if you have not committed.

```
kp = 900 |  __  __  __  __  __  __  __
kp = 600 |  __  __  __  __  __  __  __
kp = 400 |  __  __  __  __  __  __  __
kp = 200 |  __  __  __  __  __  __  __
kp = 100 |  __  __  __  __  __  __  __
kp =  50 |  __  __  __  __  __  __  __
kp =  25 |  __  __  __  __  __  __  __
         +----------------------------
   kd =     0   1   5  15  30  60 120
```

How many of the 49 will stand? Write the number down.

Run it

```
python exp1_pd_sweep.py --seconds 8
```

Six minutes or so. It prints a row at a time. Do not interrupt.

Expected:

```
sweeping 7x7 = 49 trials of 8.0s each

kp\kd        0       1       5      15      30      60     120
   25  fall1.0 fall1.0 fall1.0 DIVERGE DIVERGE DIVERGE DIVERGE
   50  fall0.9 fall1.0 fall1.0 DIVERGE DIVERGE DIVERGE DIVERGE
  100  fall1.1 fall1.1 fall1.1 DIVERGE DIVERGE DIVERGE DIVERGE
  200  fall1.3 fall1.4 fall1.4 DIVERGE DIVERGE DIVERGE DIVERGE
  400       OK      OK      OK      OK      OK DIVERGE DIVERGE
  600       OK      OK      OK      OK      OK DIVERGE DIVERGE
  900       OK      OK      OK      OK      OK DIVERGE DIVERGE

wrote /home/YOURNAME/r1_lab/exp1/exp1_pd_sweep.csv
```

15 stand. 12 topple. 22 blow up. The most common outcome is the simulator failing, not the robot.

Read the map

Three regions, and the *shape* is the finding:

- **Floor** at `kp = 400` — a horizontal edge, physical in origin.
- **Ceiling** at `kd = 15` to `60` — a **vertical** edge. Notice it does not care what `kp` is. A numerical limit ignores the physics knob.
- **The island** — 15 settings that work.

Standing is not "turn the dials until it works." It is a bounded region with two perpendicular edges that fail for entirely unrelated reasons.

Also: fall times *increase* up the left column, 0.9 s at kp=50 to 1.4 s at kp=200. Stiffer spring, smaller lean, longer to topple. That is `error = torque / kp` visible in the timing.

Look at the data you produced

```
python - <<'EOF'
import csv
for r in csv.DictReader(open("exp1_pd_sweep.csv")):
    if float(r["kd"]) == 0:
        print("kp=%4s err=%.2fdeg torque=%s"
              % (r["kp"], float(r["track_err_rad"]) * 57.3, r["peak_torque_Nm"]))
EOF
```

Expected:

```
kp=  25 err=5.15deg torque=18.0
kp=  50 err=4.41deg torque=18.2
kp= 100 err=2.85deg torque=19.9
kp= 200 err=1.74deg torque=19.8
kp= 400 err=1.67deg torque=17.6
kp= 600 err=0.99deg torque=19.2
kp= 900 err=0.60deg torque=23.3
```

Double the spring, halve the error. And the torque column never moves — 17 to 23 N·m whether it stood or fell. Nothing near a limit, ever.

> ⚠ `track_err_rad` is averaged over the **last second**. For rows that fell, the robot is already on the floor by then, so that number describes joints lying on the ground. Use the column for the **trend**, not as proof of an exact tipping point.

Draw it

Create **`exp1_plot_map.py`** in `exp1`:

```
"""Turn your exp1_pd_sweep.csv into the stability map picture.

Run it in the same folder as the CSV, after the 49-trial sweep:

    python exp1_plot_map.py

Writes exp1_map.png next to the CSV.  Three outcomes means this is a STATUS
encoding, not a categorical one, so every cell is labelled in text as well as
coloured -- the meaning never rests on colour alone.
"""
import csv, os, sys
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Patch, Rectangle

HERE = os.path.dirname(os.path.abspath(__file__))
CSV = sys.argv[1] if len(sys.argv) > 1 else os.path.join(os.getcwd(), "exp1_pd_sweep.csv")
OUT = os.path.join(os.path.dirname(CSV) or ".", "exp1_map.png")

if not os.path.exists(CSV):
    raise SystemExit(f"Could not find {CSV}\nRun the sweep first:  python exp1_pd_sweep.py --
seconds 8")

STAND, FALL, DIVERGE = "#0AA08A", "#C79A1B", "#B0473C"
INK, MUTED, PAPER = "#22252a", "#6b7078", "#fcfcfb"

rows = list(csv.DictReader(open(CSV)))
kps = sorted({float(r["kp"]) for r in rows})
kds = sorted({float(r["kd"]) for r in rows})
by = {(float(r["kp"]), float(r["kd"])): r for r in rows}

n_stand = n_fall = n_div = 0
fig, ax = plt.subplots(figsize=(9.6, 5.8), dpi=170)
fig.patch.set_facecolor(PAPER); ax.set_facecolor(PAPER)

for i, kp in enumerate(kps):
    for j, kd in enumerate(kds):
```

```
        r = by.get((kp, kd))
        if r is None:
            continue
        stood = r["stood"] == "True"
        div = r["diverged"] == "True"
        if stood:
            color, label = STAND, "STANDS"; n_stand += 1
        elif div:
            color, label = DIVERGE, "blows up"; n_div += 1
        else:
            color, label = FALL, f"falls {float(r['fall_t']):.1f}s"; n_fall += 1
        ax.add_patch(Rectangle((j + 0.03, i + 0.03), 0.94, 0.94, facecolor=color,
edgecolor="none"))
        ax.text(j + 0.5, i + 0.5, label, ha="center", va="center", color="white",
                fontsize=9.5, fontweight="bold" if stood else "normal")

ax.set_xticks([j + 0.5 for j in range(len(kds))]); ax.set_xticklabels([f"{k:g}" for k in kds],
fontsize=12)
ax.set_yticks([i + 0.5 for i in range(len(kps))]); ax.set_yticklabels([f"{k:g}" for k in kps],
fontsize=12)
ax.set_xlim(0, len(kds)); ax.set_ylim(0, len(kps))
ax.set_xlabel("kd  —  damping  (the SYRUP dial)", color=INK, fontsize=13, labelpad=10)
ax.set_ylabel("kp  —  stiffness  (the SPRING dial)", color=INK, fontsize=13, labelpad=10)
ax.tick_params(colors=MUTED, length=0)
for s in ax.spines.values():
    s.set_visible(False)

ax.text(0, len(kps) + 0.72, f"{n_stand} of {len(rows)} settings hold the robot up",
        fontsize=17, fontweight="bold", color=INK)
leg = ax.legend(handles=[Patch(facecolor=STAND, label=f"stands  ({n_stand})"),
                         Patch(facecolor=FALL, label=f"robot topples  ({n_fall})"),
                         Patch(facecolor=DIVERGE, label=f"simulation blows up  ({n_div})")],
                frameon=False, fontsize=11.5, ncol=3, loc="lower left", bbox_to_anchor=(0,
1.005))
for t in leg.get_texts():
    t.set_color(INK)

fig.tight_layout()
fig.savefig(OUT)
print(f"stands {n_stand} | topples {n_fall} | blows up {n_div}")
print(f"wrote {OUT}")
```

```
python exp1_plot_map.py
```

Expected:

```
stands 15 | topples 12 | blows up 22
wrote /home/YOURNAME/r1_lab/exp1/exp1_map.png
```

It reads **your** CSV and counts the outcomes itself — so if your numbers ever differed from this manual, the picture would show it rather than hide it. Open `exp1_map.png` from the VS Code sidebar.

Part 8 — Push it over

```
python exp1_pd_sweep.py --kp 600 --kd 5 --seconds 10 --perturb 0.15
```

Expected: `stood True`, survives the full 10 s, `max_tilt_deg 2.78`.

```
python exp1_pd_sweep.py --kp 600 --kd 5 --seconds 10 --perturb 0.20
```

Expected: `stood False`, `fall_t 2.658`, `diverged False`.

Survives 0.15 m/s. Topples at 0.20. That is slower than walking pace.

Why kd = 5 and not 30

`kd = 30` stands perfectly well — but it sits **above** the stability bound and survives only because a quiet robot has near-zero velocity. **A push injects velocity.** At `kd = 30` a hard shove destroys the *simulation* before the robot falls, and you would record the tool's limit as the robot's.

Measured, at `kp = 600`:

push	kd = 5	kd = 30
0.20	**topples at 2.66 s** (`diverged False`)	survives
0.25	topples	**`diverged True`**

> **The transferable lesson: a measurement is only valid if your tool stayed valid while you took it.** This exact mistake was in an earlier version of this experiment and went unnoticed for weeks.

Now try to fix it

Any `kp`, any `kd`. Beat 0.20 m/s with a genuine fall (`diverged False`). Five minutes.

You will not. That is the designed result.

The controller knows exactly one instruction: *get back to the pose I was told to hold.* Recovering from a shove means choosing a **different** pose — stepping, bending, shifting weight. `STAND_POS` is a constant. Nothing in this controller can decide to change it.

Not a tuning problem. A limit of what this kind of controller **is** — and the gap Lab 2 fills.

Part 9 — See it

Numbers are the result; watching is what makes them mean something.

9a — Still pictures *(works everywhere, including WSL)*

Create `lab_render.py`:

```
"""Render the expected visual outcome of each lab step (offscreen, WSL-safe).

RobotMotionViewer-style interactive viewers call mujoco.viewer.launch_passive,
which fails under WSL2 with a gladLoadGL error.  mujoco.Renderer with the
default backend works, so the lab ships still images instead of a live window.
"""
import os, sys
# WSL2 has no usable EGL; osmesa (software GL) is the backend that works here.
# Must be set before MuJoCo creates a GL context.
os.environ.setdefault("MUJOCO_GL", "osmesa")
import numpy as np, mujoco, imageio

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import exp1_pd_sweep as E

E.XML_PATH = E.resolve_xml()


def shot(kp, kd, at_s, out, perturb=0.0, W=900, H=700):
    model, data, act, jnt = E.build()
    E.set_gains(model, act, kp, kd)
    E.reset_to_stand(model, data, act, jnt)
    steps = int(at_s / model.opt.timestep)
    push_at = int(1.0 / model.opt.timestep)
    for s in range(steps):
        if perturb and s == push_at:
            data.qvel[0] += perturb
        mujoco.mj_step(model, data)
        if not np.isfinite(data.qpos).all():
            break
    # The stock scene renders almost black on a projector.  Raise the headlight
    # so the robot is actually visible in a lit room.
    model.vis.headlight.ambient[:] = [0.6, 0.6, 0.6]
    model.vis.headlight.diffuse[:] = [0.7, 0.7, 0.7]
    model.vis.headlight.specular[:] = [0.1, 0.1, 0.1]
    cam = mujoco.MjvCamera()
    mujoco.mjv_defaultCamera(cam)
    cam.distance, cam.azimuth, cam.elevation = 2.7, 135, -8
    cam.lookat[:] = [data.qpos[0], data.qpos[1], 0.5]
    # default offscreen framebuffer is 640x480; raise it before building the renderer
    model.vis.global_.offwidth = max(model.vis.global_.offwidth, W)
    model.vis.global_.offheight = max(model.vis.global_.offheight, H)
    r = mujoco.Renderer(model, height=H, width=W)   # no context-manager in 3.1.6
    try:
```

```
        r.update_scene(data, camera=cam)
        img = r.render()
    finally:
        r.close()
    imageio.imwrite(out, img)
    h = data.xpos[1][2]
    print(f"{os.path.basename(out):28s} kp={kp:<5g} kd={kd:<4g} t={at_s}s  pelvis_h={h:.3f}m")


if __name__ == "__main__":
    o = os.path.join(HERE, "lab_img")
    os.makedirs(o, exist_ok=True)
    shot(600, 30, 0.0, os.path.join(o, "01_start_pose.png"))
    shot(600, 30, 5.0, os.path.join(o, "02_standing.png"))
    shot(100, 5, 3.0, os.path.join(o, "03_toppled.png"))
    shot(600, 30, 5.0, os.path.join(o, "04_push_survived.png"), perturb=0.15)
```

```
python lab_render.py
```

Expected:

```
01_start_pose.png            kp=600   kd=30   t=0.0s  pelvis_h=0.740m
02_standing.png              kp=600   kd=30   t=5.0s  pelvis_h=0.732m
03_toppled.png               kp=100   kd=5    t=3.0s  pelvis_h=0.090m
04_push_survived.png         kp=600   kd=30   t=5.0s  pelvis_h=0.732m
```

Four PNGs in `lab_img/`. Open them in the VS Code sidebar and compare `02_standing.png` (pelvis 0.732 m) with `03_toppled.png` (0.090 m).

This draws *without a window*, which is why it works on every machine. If it fails with a GL error, run `export MUJOCO_GL=osmesa` and try once more.

9b — Watch it live *(needs a working OpenGL window — see the warning)*

Create **`exp1_watch.py`**:

```
"""Watch the R1 stand (or fall) live in the MuJoCo viewer.

    python exp1_watch.py                          # the good setting, 15 s
    python exp1_watch.py --kp 100 --kd 5          # watch it topple
    python exp1_watch.py --kd 120                 # watch the simulator blow up
    python exp1_watch.py --perturb 0.20           # shove it after 1 s
    python exp1_watch.py --loop                   # restart automatically

Controls once the window is open:
    left-drag   orbit          right-drag  pan          scroll  zoom
    space       pause/resume   backspace   reset        Esc     quit
```

```
Same physics and same control law as exp1_pd_sweep.py -- this only adds a window.
Runs in real time, so 15 simulated seconds take 15 seconds to watch.

NOTE: needs a real display and GPU. Under WSL2 or over SSH the viewer cannot open
(`gladLoadGL error`); use lab_render.py for still images there instead.
"""
import argparse, os, sys, time

import mujoco
import mujoco.viewer
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import exp1_pd_sweep as E     # reuse the model resolver, stand pose and gain setter


def run(kp, kd, seconds, perturb, loop):
    E.XML_PATH = E.resolve_xml()
    model, data, act, jnt = E.build()
    E.set_gains(model, act, kp, kd)
    E.reset_to_stand(model, data, act, jnt)

    # the stock scene is very dark; lift the headlight so it is watchable
    model.vis.headlight.ambient[:] = [0.5, 0.5, 0.5]
    model.vis.headlight.diffuse[:] = [0.6, 0.6, 0.6]

    dt = model.opt.timestep
    push_step = int(1.0 / dt)
    per_frame = 10                      # 10 physics steps per redraw -> 50 fps

    print(f"kp={kp:g}  kd={kd:g}  perturb={perturb:g}   ({seconds:g} s, real time)")
    print("space = pause, backspace = reset, Esc = quit")

    with mujoco.viewer.launch_passive(model, data) as viewer:
        viewer.cam.distance, viewer.cam.azimuth, viewer.cam.elevation = 2.8, 135, -10
        while viewer.is_running():
            E.reset_to_stand(model, data, act, jnt)
            t0, s, fell_at = time.time(), 0, None
            total = int(seconds / dt)

            while viewer.is_running() and s < total:
                frame_start = time.time()
                for _ in range(per_frame):
                    if perturb and s == push_step:
                        data.qvel[0] += perturb
                        print(f"  [{s*dt:5.2f}s]  shoved at {perturb} m/s")
                    mujoco.mj_step(model, data)
                    s += 1
                    if not np.isfinite(data.qpos).all():
                        break

                if data.warning[mujoco.mjtWarning.mjWARN_BADQACC].number > 0:
                    print(f"  [{s*dt:5.2f}s]  SIMULATION BLEW UP — not the robot, the maths")
```

```
                    break
                if fell_at is None and E.tilt_deg(data) > 45.0:
                    fell_at = s * dt
                    print(f"  [{fell_at:5.2f}s]  fell over")

                viewer.sync()
                lag = dt * per_frame - (time.time() - frame_start)
                if lag > 0:
                    time.sleep(lag)

            if fell_at is None:
                print(f"  survived the full {seconds:g} s")
            if not loop:
                print("  done — close the window to exit")
                while viewer.is_running():
                    viewer.sync()
                    time.sleep(0.05)
                break
            time.sleep(1.0)


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--kp", type=float, default=600.0)
    ap.add_argument("--kd", type=float, default=30.0)
    ap.add_argument("--seconds", type=float, default=15.0)
    ap.add_argument("--perturb", type=float, default=0.0)
    ap.add_argument("--loop", action="store_true")
    a = ap.parse_args()
    run(a.kp, a.kd, a.seconds, a.perturb, a.loop)
```

```
python exp1_watch.py
```

A real MuJoCo window. Left-drag orbits, scroll zooms, `space` pauses, `Esc` quits.

```
python exp1_watch.py --kp 100 --kd 5
```

The topple. Watch the lean develop *before* the fall.

```
python exp1_watch.py --kd 120
```

The blow-up. The robot does not fall, it **disintegrates**. Once you have seen both you will never confuse them again.

```
python exp1_watch.py --kd 5 --perturb 0.20
```

The push, going down at about 2.7 s.

> ⚠ `ERROR: gladLoadGL error` **under WSL2 is expected on some machines and is not your mistake.** It was reproduced on the machine this manual was written on, in every combination tried: passive and managed viewer, Wayland and forced X11, software rendering, GL version overrides, and with the GPU genuinely engaged. Plain GLFW windows open fine and offscreen rendering works — it is MuJoCo's interactive viewer specifically. **If you hit it, 9b and 9c are unavailable to you; use 9a, which shows the same three behaviours as pictures.** Native macOS and Linux are unaffected.

9c — Play with the dials live *(same requirement as 9b)*

Create `exp1_playground.py`:

```
"""Interactive playground: change kp / kd live and watch what happens.

    python exp1_playground.py            # explore with the arrow keys
    python exp1_playground.py --tour     # auto-cycle all 49 settings, 4 s each
    python exp1_playground.py --record demo.mp4    # record a scripted session
                                                   # (works headless / WSL)

KEYS (click the 3-D view first so it has focus)
    UP / DOWN     stiffer / softer spring   (kp)
    RIGHT / LEFT  more / less syrup         (kd)
    R             restart the current setting
    P             shove it, 0.20 m/s, right now
    T             toggle the automatic tour
    Esc           quit

Every change resets the robot to the same starting pose, so what you see is
caused by the dials and nothing else. The console reports the outcome of each
setting as it happens.

Needs a real display -- under WSL2 or over SSH use lab_render.py instead.
"""
import argparse, os, sys, time

import mujoco
import mujoco.viewer
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import exp1_pd_sweep as E

KPS = [25, 50, 100, 200, 400, 600, 900]
KDS = [0, 1, 5, 15, 30, 60, 120]
UP, DOWN, RIGHT, LEFT, KEY_R, KEY_P, KEY_T = 265, 264, 262, 263, 82, 80, 84


class State:
```

```
    def __init__(self, tour):
        self.i = KPS.index(600)     # kp index
        self.j = KDS.index(30)      # kd index
        self.restart = True
        self.push = False
        self.tour = tour

    @property
    def kp(self):
        return KPS[self.i]

    @property
    def kd(self):
        return KDS[self.j]

    def advance_tour(self):
        self.j += 1
        if self.j >= len(KDS):
            self.j = 0
            self.i = (self.i + 1) % len(KPS)
        self.restart = True


def apply_key(st, k):
    """The one place a keypress changes anything.  Shared by the live viewer
    and by --record, so a recorded session is driven by exactly the same code
    a student's fingers drive."""
    if k == UP:      st.i = min(st.i + 1, len(KPS) - 1); st.restart = True
    elif k == DOWN:  st.i = max(st.i - 1, 0);            st.restart = True
    elif k == RIGHT: st.j = min(st.j + 1, len(KDS) - 1); st.restart = True
    elif k == LEFT:  st.j = max(st.j - 1, 0);            st.restart = True
    elif k == KEY_R: st.restart = True
    elif k == KEY_P: st.push = True
    elif k == KEY_T: st.tour = not st.tour; print(f"  tour {'ON' if st.tour else 'OFF'}")


# A scripted session for --record: (key pressed, seconds to watch, caption).
# This is what a student does in their first two minutes: walk the spring down
# until it topples, come back, push the damping up until the simulator dies,
# then shove the robot.
DEMO = [
    (None,   3.0, "start here"),
    (LEFT,   0.0, "less damping"),
    (LEFT,   2.5, "less damping - still stands"),
    (DOWN,   3.0, "softer spring"),
    (DOWN,   3.5, "softer again - watch the lean"),
    (DOWN,   3.0, "softer again"),
    (UP,     0.0, "back up"),
    (UP,     0.0, "back up"),
    (UP,     2.0, "back to a setting that stands"),
    (RIGHT,  0.0, "more damping"),
    (RIGHT,  0.0, "more damping"),
    (RIGHT,  2.5, "more damping"),
    (LEFT,   0.0, "back off"),
```

```
    (LEFT,   0.0, "back off"),
    (LEFT,   1.5, "back to kd = 5"),
    (KEY_P,  4.0, "shove it, 0.20 m/s"),
]


def _font(size, bold=False):
    """A real TrueType face if one can be found -- the PIL default bitmap font
    is unreadable on a projector."""
    from PIL import ImageFont
    import glob
    names = ["DejaVuSansMono-Bold.ttf" if bold else "DejaVuSansMono.ttf",
             "DejaVuSans-Bold.ttf" if bold else "DejaVuSans.ttf"]
    roots = ["/usr/share/fonts", "C:/Windows/Fonts",
             os.path.join(sys.prefix, "lib")]
    for n in names:
        for root in roots:
            hit = glob.glob(os.path.join(root, "**", n), recursive=True)
            if hit:
                try:
                    return ImageFont.truetype(hit[0], size)
                except OSError:
                    pass
    return ImageFont.load_default()


def _hud(img, st, caption, outcome, t, seconds):
    """Burn the dial values and the outcome into the frame."""
    from PIL import Image, ImageDraw
    im = Image.fromarray(img)
    d = ImageDraw.Draw(im, "RGBA")
    big, mid, small = _font(30, True), _font(17), _font(16)
    d.rectangle([0, 0, im.width, 96], fill=(16, 22, 32, 220))
    d.text((20, 12), f"kp = {st.kp}", font=big, fill=(255, 255, 255))
    d.text((20, 52), f"kd = {st.kd}", font=big, fill=(255, 255, 255))
    d.text((228, 16), "arrow keys change the dials, live",
           font=small, fill=(150, 200, 235))
    d.text((228, 42), f"just pressed:  {caption}", font=mid,
           fill=(245, 200, 120))
    d.text((228, 68), f"t = {min(t, seconds):.1f}s", font=small,
           fill=(160, 172, 188))
    if outcome:
        colour = ((245, 120, 100) if "fell" in outcome else
                  (255, 205, 90) if "BLEW" in outcome else (140, 225, 175))
        d.rectangle([0, im.height - 44, im.width, im.height],
                    fill=(16, 22, 32, 220))
        d.text((20, im.height - 34), outcome, font=mid, fill=colour)
    return np.asarray(im)


def record(path, seconds, fps=30, width=720, height=540):
    """Run the DEMO script offscreen and write an mp4.  Same physics, same
    keys, same outcome logic as the live viewer -- only the display differs."""
    import imageio.v2 as imageio
```

```
    E.XML_PATH = E.resolve_xml()
    model, data, act, jnt = E.build()
    model.vis.headlight.ambient[:] = [0.5, 0.5, 0.5]
    model.vis.headlight.diffuse[:] = [0.6, 0.6, 0.6]
    dt = model.opt.timestep
    st = State(False)

    model.vis.global_.offwidth = width       # default framebuffer is 640x480
    model.vis.global_.offheight = height
    renderer = mujoco.Renderer(model, height=height, width=width)
    cam = mujoco.MjvCamera()
    mujoco.mjv_defaultCamera(cam)
    cam.distance, cam.azimuth, cam.elevation = 2.5, 135, -8
    cam.lookat[:] = [0, 0, 0.55]

    frames = []
    step_per_frame = max(1, int(round(1.0 / fps / dt)))
    s, outcome = 0, None
    for key, hold, caption in DEMO:
        if key is not None:
            apply_key(st, key)
        if hold <= 0:                        # chorded presses: no pause between
            continue
        if st.restart:
            E.set_gains(model, act, st.kp, st.kd)
            E.reset_to_stand(model, data, act, jnt)
            st.restart = False
            s, outcome = 0, None
        print(f"kp={st.kp:<4g} kd={st.kd:<4g}  {caption}")
        for _ in range(int(hold * fps)):
            for _ in range(step_per_frame):
                if st.push:
                    data.qvel[0] += 0.20
                    st.push = False
                mujoco.mj_step(model, data)
                s += 1
            if outcome is None:
                if data.warning[mujoco.mjtWarning.mjWARN_BADQACC].number > 0:
                    outcome = "BLEW UP - the simulator, not the robot"
                    print(f"    {outcome}")
                elif E.tilt_deg(data) > 45.0:
                    outcome = f"fell at {s * dt:.2f}s"
                    print(f"    {outcome}")
                elif s * dt >= 2.4:
                    outcome = "still standing"
            renderer.update_scene(data, cam)
            frames.append(_hud(renderer.render(), st, caption, outcome,
                               s * dt, seconds))
    imageio.mimsave(path, frames, fps=fps, quality=8)
    print(f"\nwrote {path}  ({len(frames)} frames, {len(frames) / fps:.1f}s)")


def main(seconds, tour):
```

```
E.XML_PATH = E.resolve_xml()
model, data, act, jnt = E.build()
model.vis.headlight.ambient[:] = [0.5, 0.5, 0.5]
model.vis.headlight.diffuse[:] = [0.6, 0.6, 0.6]
dt = model.opt.timestep
st = State(tour)

def on_key(k):
    apply_key(st, k)

print(__doc__.split("KEYS")[1].split("Every change")[0])
print("start:  kp=600  kd=30   (arrow keys to change)\n")

with mujoco.viewer.launch_passive(model, data, key_callback=on_key) as viewer:
    viewer.cam.distance, viewer.cam.azimuth, viewer.cam.elevation = 2.8, 135, -10
    s = 0
    outcome = None
    t_setting = time.time()

    while viewer.is_running():
        if st.restart:
            E.set_gains(model, act, st.kp, st.kd)
            E.reset_to_stand(model, data, act, jnt)
            s, outcome, st.restart = 0, None, False
            t_setting = time.time()
            print(f"kp={st.kp:<4g} kd={st.kd:<4g}  ...", end="", flush=True)

        frame = time.time()
        for _ in range(10):
            if st.push:
                data.qvel[0] += 0.20
                st.push = False
                print(" [shoved 0.20]", end="", flush=True)
            mujoco.mj_step(model, data)
            s += 1

        if outcome is None:
            if data.warning[mujoco.mjtWarning.mjWARN_BADQACC].number > 0:
                outcome = "BLEW UP  (the simulator, not the robot)"
                print(f"  {outcome}", flush=True)
            elif E.tilt_deg(data) > 45.0:
                outcome = f"fell at {s*dt:.2f}s"
                print(f"  {outcome}", flush=True)
            elif s * dt >= seconds:
                outcome = f"STOOD the full {seconds:g}s"
                print(f"  {outcome}", flush=True)

        viewer.sync()
        lag = dt * 10 - (time.time() - frame)
        if lag > 0:
            time.sleep(lag)

        if st.tour and (outcome is not None) and (time.time() - t_setting > 2.0):
            st.advance_tour()
```

```
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--seconds", type=float, default=8.0)
    ap.add_argument("--tour", action="store_true")
    ap.add_argument("--record", metavar="OUT.MP4",
                    help="record a scripted session offscreen instead of "
                         "opening a window (works on WSL / headless)")
    a = ap.parse_args()
    if a.record:
        record(a.record, a.seconds)
    else:
        main(a.seconds, a.tour)
```

```
python exp1_playground.py
```

Click the 3-D view first, then:

Key	Effect
↑ / ↓	stiffer / softer spring
← / →	less / more syrup
R	restart this setting
P	shove it, 0.20 m/s, now
T	toggle the automatic tour

Every change resets to the same starting pose, so what you see is caused by the dials and nothing else. Add `--tour` to walk all 49 automatically — **your CSV, as a movie.**

Two things to try: walk down the `kp` column at `kd = 5` and watch the lean grow before each fall. Then sit at `kp = 600` and walk right along `kd` — 0, 1, 5, 15, 30 all look identical, then 60 detonates with no warning. That abruptness *is* the signature of a numerical failure.

Part 10 — Change the code

Create `exp1_sandbox.py`:

```
"""SANDBOX — edit the marked zones below, save, re-run, watch what changes.

    python exp1_sandbox.py              # watch it in the MuJoCo viewer
    python exp1_sandbox.py --headless   # no window, just the verdict (works over SSH / WSL)

Everything you are meant to touch is in the five EDIT ZONES below. The machinery
underneath is at the bottom and you can ignore it.
```

```
Workflow:  change one thing  ->  save  ->  re-run  ->  watch  ->  write down what happened.
One change at a time. Two changes at once and you cannot attribute the result.
"""
import argparse, os, sys, time
import numpy as np
import mujoco

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
try:
    import exp1_pd_sweep as E
except ModuleNotFoundError:
    raise SystemExit(
        "Could not find exp1_pd_sweep.py.\n"
        f"This sandbox must sit in the SAME folder as it. It is currently in:\n  {HERE}\n"
        "Copy exp1_sandbox.py next to exp1_pd_sweep.py and run it from there.")


# ============================================================================
# EDIT ZONE 1 — the pose you ask the robot to hold   (this is q* in the formula)
# ============================================================================
# Angles are RADIANS. 0.1 rad ≈ 5.7°.  Negative hip/ankle pitch leans it forward.
#
# TRY:  knee 0.3 -> 0.6            a deeper crouch. Does the stable island widen?
#       ankle_pitch -0.2 -> -0.35  lean it back. Does it survive a bigger shove?
#       hip_pitch -0.1 -> 0.0      stand up straight. Watch what the ankles do.
STAND_POS = {
    "left_hip_pitch": -0.1, "left_hip_roll": 0.0, "left_hip_yaw": 0.0,
    "left_knee": 0.3, "left_ankle_pitch": -0.2, "left_ankle_roll": 0.0,
    "right_hip_pitch": -0.1, "right_hip_roll": 0.0, "right_hip_yaw": 0.0,
    "right_knee": 0.3, "right_ankle_pitch": -0.2, "right_ankle_roll": 0.0,
}

# ============================================================================
# EDIT ZONE 2 — the two dials, and per-joint overrides
# ============================================================================
KP = 600.0        # spring: torque per radian of error
KD = 5.0          # damper: torque per rad/s of speed
#
# WHY THE DEFAULT IS 5 AND NOT 30, even though both stand:
#   kd = 30 sits ABOVE the ~11 stability bound and survives only because a quiet
#   standing robot has near-zero velocity. Change ANYTHING else -- mass, friction,
#   your own control law -- and it tips into a numerical blow-up that hides the
#   effect you were trying to see. Measured, at kd=30 vs kd=5:
#       mass x1.5        blew up 7.6s   ->   fell 8.60s
#       friction + push  blew up 1.0s   ->   stood
#       knee sine wave   blew up 0.4s   ->   fell 1.32s
#   kd = 5 is inside the unconditionally stable zone. Explore from there.

# The sweep uses ONE gain for all 12 joints. Real robots don't. Override here:
#
# TRY:  KP = 200 plus {"left_ankle_pitch": 900, "right_ankle_pitch": 900}
#         VERIFIED: this STANDS, even though kp=200 everywhere falls at 1.4s.
```

```
#         Stiffening two joints out of twelve rescues it — which proves the ankle
#         is the joint that was failing. Direct evidence for the whole A5 argument.
#       {"left_knee": 150, "right_knee": 150}
#         soft knees — where does it sag first?
PER_JOINT_KP = {}
PER_JOINT_KD = {}

# ============================================================================
# EDIT ZONE 3 — the world
# ============================================================================
GRAVITY = -9.81      # TRY: -1.62 (moon) with KP = 200.
                     #   VERIFIED: stands. kp=200 fails on Earth. Less weight to hold
                     #   means less permanent sag -- exactly what error = torque/kp says.
MASS_SCALE = 1.0     # TRY: 1.5.  VERIFIED at kd=5: falls at 8.60s. Heavier robot,
                     #   more torque needed, more sag. Now find the kp that saves it.
FRICTION = None      # TRY: 0.2 (icy). VERIFIED: stands, and still survives a 0.15 push.
                     #   Standing still asks very little of friction. Try it with PUSH.
TIMESTEP = 0.002     # ** THE BEST EXPERIMENT IN THIS FILE **
                     #   Set KD = 60 and leave dt at 0.002  ->  blows up at 0.06s
                     #   Now set TIMESTEP = 0.001            ->  STANDS the full 10s
                     #   VERIFIED, both. Nothing about the robot changed. You have just
                     #   proven that failure belonged to the simulator, not the machine.

# ============================================================================
# EDIT ZONE 4 — the trial
# ============================================================================
SECONDS = 10.0
PUSH = 0.0           # sideways shove at t = 1 s, in m/s. TRY 0.15, then 0.20.

# ============================================================================
# EDIT ZONE 5 — write your own control law   (advanced, and the interesting one)
# ============================================================================
# `ctrl` holds the 12 position TARGETS the PD law is chasing. By default they are
# constant — that constancy is exactly why this controller cannot recover from a
# shove, and it is the whole point of Experiment 1.
#
# Here you can make the target change over time. Return the modified ctrl.
#
# TRY 1 — make it move, and watch how little it takes to topple it:
#     a = 0.05 * np.sin(2 * np.pi * 0.5 * t)
#     ctrl[names["left_knee"]]  = 0.3 + a
#     ctrl[names["right_knee"]] = 0.3 + a
#     VERIFIED: falls at 2.66s. And that is not a bad edit -- it is the result.
#     I tried knees, hip yaw and ankle roll, at amplitudes from 0.25 down to 0.05
#     rad, and EVERY commanded motion topples this robot. It has so little margin
#     that simply moving on purpose is enough to lose it.
#     Finding a motion it CAN survive is a genuinely good exercise. Good luck.
#
# TRY 2 — the bridge to Experiment 2. React to the robot's state:
#     tilt = data.qpos[2]                      # base height, drops when falling
#     if t > 1.0 and tilt < 0.70:
#         ctrl[names["left_knee"]] += 0.3      # crouch when you start to go
#         ctrl[names["right_knee"]] += 0.3
#     Now the target is no longer constant. Can you survive a shove the fixed
```

```
#     controller cannot? That is precisely what a learned policy does — except it
#     works out the rule instead of you writing it.
def custom_control(t, data, ctrl, names):
    return ctrl


# ═══════════════════════════════════════════════════════════════════════
#  machinery below — you do not need to change any of this
# ═══════════════════════════════════════════════════════════════════════
def build_world():
    E.XML_PATH = E.resolve_xml()
    E.STAND_POS = STAND_POS                      # sandbox pose wins
    model, data, act, jnt = E.build()

    model.opt.timestep = TIMESTEP
    model.opt.gravity[:] = [0.0, 0.0, GRAVITY]
    if MASS_SCALE != 1.0:
        model.body_mass[:] *= MASS_SCALE
        model.body_inertia[:] *= MASS_SCALE
    if FRICTION is not None:
        model.geom_friction[:, 0] = FRICTION

    E.set_gains(model, act, KP, KD)
    for name, v in PER_JOINT_KP.items():
        i = act[name]; model.actuator_gainprm[i, 0] = v; model.actuator_biasprm[i, 1] = -v
    for name, v in PER_JOINT_KD.items():
        model.actuator_biasprm[act[name], 2] = -v

    E.reset_to_stand(model, data, act, jnt)
    return model, data, act, jnt


def report():
    bits = [f"kp={KP:g} kd={KD:g}", f"gravity={GRAVITY:g}", f"dt={TIMESTEP:g}"]
    if PER_JOINT_KP: bits.append(f"per-joint kp={PER_JOINT_KP}")
    if MASS_SCALE != 1.0: bits.append(f"mass x{MASS_SCALE:g}")
    if FRICTION is not None: bits.append(f"friction={FRICTION:g}")
    if PUSH: bits.append(f"push={PUSH:g}")
    print(" | ".join(bits))


def step_block(model, data, act, s, n=10):
    """Advance n physics steps, applying the custom control law each step."""
    for _ in range(n):
        t = s * model.opt.timestep
        if PUSH and s == int(1.0 / model.opt.timestep):
            data.qvel[0] += PUSH
        data.ctrl[:] = custom_control(t, data, data.ctrl.copy(), act)
        mujoco.mj_step(model, data)
        s += 1
    return s


def verdict(model, data, s):
```

```
    if data.warning[mujoco.mjtWarning.mjWARN_BADQACC].number > 0:
        return f"BLEW UP at {s*model.opt.timestep:.3f}s  (simulator, not robot)"
    if E.tilt_deg(data) > 45.0:
        return f"fell at {s*model.opt.timestep:.2f}s"
    return None


def run_headless(model, data, act, jnt):
    s, total = 0, int(SECONDS / model.opt.timestep)
    while s < total:
        s = step_block(model, data, act, s)
        v = verdict(model, data, s)
        if v:
            print("  " + v); return
    print(f"  STOOD the full {SECONDS:g}s   (max tilt {E.tilt_deg(data):.2f}°)")


def run_viewer(model, data, act, jnt):
    import mujoco.viewer
    model.vis.headlight.ambient[:] = [0.5, 0.5, 0.5]
    model.vis.headlight.diffuse[:] = [0.6, 0.6, 0.6]
    dt = model.opt.timestep
    with mujoco.viewer.launch_passive(model, data) as viewer:
        viewer.cam.distance, viewer.cam.azimuth, viewer.cam.elevation = 2.8, 135, -10
        s, done, total = 0, None, int(SECONDS / dt)
        while viewer.is_running():
            frame = time.time()
            if s < total and done is None:
                s = step_block(model, data, act, s)
                done = verdict(model, data, s)
                if done:
                    print("  " + done)
                elif s >= total:
                    done = f"STOOD the full {SECONDS:g}s"
                    print("  " + done)
            viewer.sync()
            lag = dt * 10 - (time.time() - frame)
            if lag > 0:
                time.sleep(lag)


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--headless", action="store_true")
    a = ap.parse_args()
    report()
    m, d, act, jnt = build_world()
    (run_headless if a.headless else run_viewer)(m, d, act, jnt)
```

```
python exp1_sandbox.py --headless
```

Expected:

```
kp=600 kd=5 | gravity=-9.81 | dt=0.002
  STOOD the full 10s   (max tilt 0.48°)
```

A file built to be edited. Five marked zones; machinery at the bottom. `--headless` gives a verdict with no window, which is the mode to use if 9b failed for you. **It must sit next to `exp1_pd_sweep.py`** — it imports from it.

Zone	What you control
1	the pose it holds
2	the two dials, **including per-joint overrides**
3	gravity, mass, friction, timestep
4	trial length and push
5	**your own control law**

Three experiments with verified answers

The best one in the lab. Set `KD = 60`, leave `TIMESTEP = 0.002` → blows up at 0.06 s. Now set `TIMESTEP = 0.001` → **stands the full 10 s.** Nothing about the robot changed. You have just proven that failure belonged to the simulator.

Prove which joint was failing. `KP = 200` everywhere falls at 1.4 s. Add `PER_JOINT_KP = {"left_ankle_pitch": 900, "right_ankle_pitch": 900}` → **stands.** Stiffening two joints out of twelve rescues it. The ankle was the problem all along.

Moon gravity with `KP = 200` → **stands**, though 200 fails on Earth. Less weight to hold, less sag. `error = torque / kp`, made visible.

Why the sandbox defaults to kd = 5

Both 5 and 30 stand. But 30 is only *conditionally* stable, so changing anything else tips it into a blow-up that hides the effect you were studying:

change	at kd = 30	at kd = 5
mass ×1.5	blew up 7.6 s	falls 8.60 s
icy floor + push	blew up 1.0 s	stands
knee sine wave	blew up 0.4 s	falls 1.32 s

A sandbox default must be numerically robust, not merely a setting that stands.

Zone 5 — the hard one

Make the target move and the robot falls. Measured: knees, hip yaw and ankle roll, amplitudes from 0.25 down to 0.05 rad — **every commanded motion topples it**, the gentlest at 2.66 s.

That is the result, not a mistake. This controller has so little margin that moving on purpose loses it. Writing a rule that reacts to the robot's state and *helps* is genuinely hard.

Which is exactly what Lab 2 is about — a learned policy works out that rule instead of you writing it, and survives twice the push.

Part 11 — Delete it and build it again

The result is not the point. The workflow is.

```
cd ~
rm -rf ~/r1_lab/exp1
```

Yes, really. Your conda environment and the `unitree_mujoco` clone survive — only the experiment folder is gone.

Now rebuild it from Part 1 to Part 5, without reading the prose. Ten minutes if you understood what you did the first time, and if you did not, the place you get stuck is precisely the thing worth asking about.

```
mkdir -p ~/r1_lab/exp1/model/assets
cd ~/r1_lab/exp1
cp ~/r1_lab/unitree_mujoco/unitree_robots/r1/meshes/* model/assets/
```

then re-create `model/r1_standalone.xml`, `check_model.py` and `exp1_pd_sweep.py`, and finish with:

```
python check_model.py
python exp1_pd_sweep.py --kp 600 --kd 30 --seconds 5
```

> ☑ `total mass 28.93 kg` **and** `stood True` **a second time, from an empty folder.** That is the lab.

Troubleshooting

Symptom	Cause & fix	
`conda: command not found` right after installing	you are in the shell that existed before `conda init`. Close it, open a new one	
Prompt does not say `(r1lab)`	`conda activate r1lab` — needed in every new terminal	
`ModuleNotFoundError: mujoco`	the environment is not active; see the row above	

Symptom	Cause & fix	
`ModuleNotFoundError: imageio`	0f installed only three libraries — rerun it with `imageio` on the end	
`ls model/assets \`	wc -l` prints 0	the `cp` in Part 2 ran from the wrong folder — `cd ~/r1_lab/unitree_mujoco` first
`resource not found ... pelvis_link.STL`	same cause: the meshes are not in `model/assets`	
`XML Error: ... unrecognized` on your new file	the paste is truncated. The file is 378 lines — check with `wc -l model/r1_standalone.xml`	
`code: command not found` (Windows)	the **WSL** extension is not installed on the Windows side	
`git clone` is slow or stalls	drop `--filter=blob:none`; it costs more download but fails less on restricted networks	
`ERROR: gladLoadGL error`	no usable OpenGL window. Expected under WSL2 on some machines — use Part 9a	
`unrecognized arguments: --push`	the flag is `--perturb`	
`Could not find exp1_pd_sweep.py`	the sandbox must live beside it, in `exp1`	
All results `nan`	`kd` too high — that is Part 6b, working as intended	
Sweep much slower than 6 min	use `--seconds 5`; the island does not move	
Numbers differ from this manual	**tell the instructor — that is a finding**	

What to hand in

1. The output of `check_model.py` from **your** machine.
2. Your prediction grid from Part 7, filled in **before** running.
3. Your `exp1_pd_sweep.csv` and `exp1_map.png`.
4. One paragraph: **how do you tell a topple from a blow-up?** Name the field.
5. The highest push you survived, with gains, and evidence `diverged` was `False`.
6. One sandbox experiment you ran, what you changed, what you expected, what happened.
7. One sentence: **what can this controller never do, and why?**
8. **Name three things in `model/r1_standalone.xml` that are not in the file you downloaded from Unitree, and say why each had to be added.**

Appendix A — Background, Files, and Code

Read Part A before the lab. Parts B and C go with Parts 5 and 6, when you first open the files themselves.

> **A note on the maths.** There is one formula in this experiment and everything else follows from it. You do not need it to *run* the lab — but you need it to understand why the results look the way they do, and you will not be able to answer "why?" without it.

PART A — The physics

A1. The problem: you cannot tell a motor where to be

A motor produces **torque** — a twisting effort, measured in newton-metres (N·m). That is the only thing it can do. There is no command that means "be at 30 degrees."

So how does a robot hold a pose? You invent a rule that converts *where I want to be* into *how hard to twist.* The simplest useful rule is called **PD control**, and it is the entire subject of this experiment.

A2. The formula

For one joint:

```
τ  =  kp · (q* − q)  −  kd · q̇
```

Symbol	Meaning	Units
`τ`	torque the motor applies	N·m
`q`	where the joint actually is	radians
`q*`	where you want it (the target)	radians
`q̇`	how fast the joint is moving	rad/s
`kp`	**stiffness** — the spring dial	N·m per radian
`kd`	**damping** — the syrup dial	N·m per (rad/s)

Two terms, two jobs.

`kp · (q* − q)` **— the spring.** The further the joint is from target, the harder it is pulled back. This is Hooke's law. Double `kp` and you double the restoring torque for the same error.

`−kd · q̇` **— the damper.** It opposes *motion itself*, regardless of position. This is a shock absorber. Without it the spring pulls the joint to the target, overshoots, pulls back, overshoots — forever.

The analogy that carries all the way through: **`kp` is the spring in a screen door, `kd` is syrup packed into the hinge.** Weak spring and the door never shuts. No syrup and it slams and bounces. Too much syrup and it jams.

A3. Twelve joints, twelve different torques, from two numbers

This is the part worth pausing on.

You set **one** `kp` and **one** `kd`, and they are applied to **all twelve leg joints**. But every joint produces a *different* torque, because every joint has a different error `(q* − q)` and a different speed $\dot{q}$ at that instant.

A concrete example. Suppose `kp = 600`:

Joint	error `q* − q`	torque `kp · error`
left knee	0.004 rad	2.4 N·m
left ankle pitch	0.016 rad	9.6 N·m
left hip roll	0.000 rad	0.0 N·m

Same dial, three different torques — because the load on each joint differs. The ankle is carrying the whole robot's weight through a long lever; the hip roll is barely loaded at all.

So the two dials do not set torque. **They set a *relationship* between error and torque**, and physics decides the rest.

A4. How joint torque becomes balance

Standing is not about joints. It is about where your **weight line** falls.

Draw a vertical line down from the robot's centre of mass. If it lands inside the area covered by the feet — the **support polygon** — the robot is stable. If it falls outside, the robot rotates about the edge of its foot and goes over. Nothing can stop it except moving a foot.

The ankle is the joint that controls this. Ankle torque shifts the **centre of pressure**, the point on the sole where the ground pushes back. Push harder with the toes and the pressure point moves forward; push with the heel and it moves back. That is how a standing robot — or you — makes tiny corrections without visibly moving.

Chain of causation for this whole experiment:

```
kp, kd  →  torque at each joint  →  ankle torque  →  centre of pressure
       →  weight line inside or outside the foot  →  stands or falls
```

A5. Why weak stiffness *must* fail — the floor

Look at the formula again. If the joint is exactly at target, the error is zero, so the torque is zero. But gravity is still pulling.

So to hold **any** load, a joint has to sit at some non-zero error. Rearranging:

```
steady-state error  =  τ_required / kp
```

That error is not a wobble that settles out. **It is the equilibrium.** The joint parks there and stays.

The loaded ankle on this robot must hold roughly **9.6 N·m**. So:

kp	permanent ankle error
100	0.096 rad ≈ 5.5°
200	0.048 rad ≈ 2.8°
400	0.024 rad ≈ 1.4°
900	0.011 rad ≈ 0.6°

A permanent lean of a few degrees walks the weight line toward the toe. Past a threshold it leaves the foot and the robot tips.

> **The counter-intuitive part**
>
> The peak torque available is about **31 N·m** — over three times what the task needs. In the sweep, the settings that *fell* used **less** torque than the ones that stood.
>
> **The robot does not fall because it is weak. It falls because it is leaning.** This is a balance failure, not a strength failure, and it is the single most important idea in the lab.

A6. Why heavy damping *must* fail — the ceiling

This one has nothing to do with robotics. It is arithmetic on a grid.

The simulator does not flow continuously. It advances in discrete steps of `dt = 0.002 s`. Within a step, the damping term is applied as a fixed correction. Roughly, each step multiplies the joint's velocity by:

```
1 − (kd · dt / M)
```

where `M` is the joint's effective inertia. Three regimes:

Factor	Behaviour
between 0 and 1	velocity shrinks smoothly — **settles**
between −1 and 0	flips sign but shrinks — oscillates, still settles
below −1	flips sign **and grows** — **runs away**

Stability therefore needs:

```
kd  <  2M / dt
```

The **lightest joint sets the limit for the whole robot**, because one runaway joint injects energy into a connected chain. On this robot the ankle roll link is a small piece of metal at the end of the leg, and it caps `kd` at roughly **11**.

> **This ceiling is a property of your simulator, not of the R1.** Halve `dt` and the unstable region shrinks. A real robot has no such limit — it has different ones. Recognising which failures are physics and which are numerics is a genuinely useful skill, and this experiment gives you both in one afternoon.

A7. So what shape should the results have?

Predict before you run:

- **Too little** `kp` → permanent lean → topples. A floor.
- **Too much** `kd` → numerical blow-up. A ceiling.
- **In between** → it stands.

Standing should live on an **island**, not in a corner. That is exactly what the 49-trial sweep shows.

PART B — The files

B1. What is an XML model?

MuJoCo describes a robot in a plain-text file called **MJCF**, which is XML. It is human-readable — open it in any editor. It declares:

- **bodies** — pelvis, thigh, shin, foot, and how they connect
- **joints** — where bodies rotate relative to each other, and their limits
- **geoms** — the shapes used for collision and for display
- **actuators** — the motors, and their control law
- **options** — global settings such as the timestep

Opening `model/r1_standalone.xml`, the first line tells you two important things:

```
<compiler angle="radian" meshdir="assets"/>
```

- `angle="radian"` — **every angle in this file is radians, not degrees.** 0.3 rad ≈ 17°.
- `meshdir="assets"` — mesh files are looked up in a folder called `assets` **next to this XML**. This is why the model cannot travel alone.

Further down you find the actuators:

```
<position name="left_hip_pitch" joint="left_hip_pitch_joint" kp="600" dampratio="1"/>
<position name="left_ankle_pitch" joint="left_ankle_pitch_joint" kp="300" dampratio="1"/>
```

A `<position>` actuator **is** a PD controller — MuJoCo implements exactly the formula from A2. The XML ships sensible per-joint defaults (600 for hips and knees, 300 for ankle pitch).

The experiment overrides all of them with a single uniform `kp` and `kd`, which is what makes a two-dial sweep possible. Worth knowing: real robots are usually tuned per joint, and this deliberately does not.

B2. What is a mesh?

A **mesh** is a 3-D shape stored as a list of triangles — a `.STL` file. It is the actual physical form of a link: the curve of a thigh, the sole of a foot.

The XML does not contain shapes. It contains **references** to them:

```
<mesh file="pelvis_link.STL"/>
```

MuJoCo looks that up inside `meshdir`, i.e. `model/assets/`. There are **43** of them, about 22 MB. If you copy the XML without the `assets` folder you get:

```
resource not found via provider or OS filesystem: 'pelvis_link.STL'
```

That is not a corrupt install. It is a missing shape.

Meshes serve two different purposes here — pretty shapes for *display*, and simplified shapes for *collision*. Files named `..._collision.STL` are the ones physics actually uses; collision maths on a full detailed mesh would be far too slow.

B3. What is in your folder

You created every one of these. Nothing was extracted from an archive: the meshes came from Unitree's public repository in Part 2, and every other file was typed or pasted into existence in Parts 3 to 10. Nothing here reaches outside this folder.

```
exp1/
├── check_model.py        proves the model loads — Part 4
├── exp1_pd_sweep.py      the experiment — Parts 5 to 8
├── exp1_sandbox.py       the file you edit — Part 10
├── exp1_watch.py         the live 3-D viewer — Part 9b
├── exp1_playground.py    the live viewer with arrow-key dials — Part 9c
├── exp1_plot_map.py      draws your stability map from your CSV — Part 7
├── lab_render.py         saves still pictures of the robot — Part 9a
└── model/
    ├── r1_standalone.xml the robot description — you wrote this, Part 3
    └── assets/           43 mesh files — from Unitree, Part 2
```

File	What it is	Do you run it?
`exp1_pd_sweep.py`	240 lines of Python. Sets gains, runs trials, records results	**Yes — this is the lab**
`check_model.py`	17 lines. Loads the model and prints what MuJoCo found	**Yes — Part 4**
`exp1_sandbox.py`	A copy built to be edited: five marked zones	**Yes — Part 10**
`exp1_watch.py`	Opens a real MuJoCo window and runs one setting	Part 9b
`exp1_playground.py`	The same window, with the dials on the arrow keys	Part 9c
`exp1_plot_map.py`	Turns your own `exp1_pd_sweep.csv` into a picture	Part 7
`lab_render.py`	Renders still images — works with no display	Part 9a

File	What it is	Do you run it?
`r1_standalone.xml`	The robot: bodies, joints, motors, timestep	No — it is data
`assets/*.STL`	The physical shapes	No — they are data

Two files appear as you work, in this same folder: `exp1_pd_sweep.csv` (Part 7) and `exp1_map.png` (Part 7), plus a `lab_img/` folder of pictures if you run Part 9a. Those are your results.

PART C — Code walkthrough: `exp1_pd_sweep.py`

Read alongside the file. Line numbers are approximate but the section names are exact.

C1. Finding the model — `resolve_xml()` *(lines ~39–72)*

Tries several locations in order and returns **the first that actually loads**, rather than the first that exists.

Why that distinction matters: an earlier version of this project had a stray copy of the XML with no `assets` folder beside it. An existence check passed, then MuJoCo died on `pelvis_link.STL`. Attempting the load is the only honest test.

C2. The target pose — `STAND_POS` *(lines 78–84)*

```
STAND_POS = {
    "left_hip_pitch": -0.1, "left_knee": 0.3, "left_ankle_pitch": -0.2,
    ...
}
```

This is `q*` from the formula — the pose every joint is asked to hold, in radians. A slight crouch: hips back 0.1 rad (≈6°), knees bent 0.3 rad (≈17°), ankles compensating −0.2 rad.

It is a **constant**. It never changes during a trial. That fact is the entire reason this controller cannot recover from a shove — there is no mechanism for choosing a different pose.

C3. The heart of it — `set_gains()` *(lines 96–102)*

```
def set_gains(model, act, kp, kd):
    for i in act.values():
        model.actuator_gainprm[i, 0] = kp
        model.actuator_biasprm[i, 1] = -kp
        model.actuator_biasprm[i, 2] = -kd
```

Three lines are the whole experiment.

MuJoCo stores each actuator's control law as coefficients. For a `<position>` actuator they encode exactly $\tau = kp \cdot (q^* - q) - kd \cdot \dot{q}$:

Array slot	Holds	Why the sign
`gainprm[i,0]`	`+kp`	multiplies the target

Array slot	Holds	Why the sign
`biasprm[i,1]`	`-kp`	subtracts the current position → gives `kp·(q*-q)`
`biasprm[i,2]`	`-kd`	subtracts velocity → gives `-kd·q̇`

These are writable **after** the model compiles, so 49 trials take six minutes instead of an afternoon of editing XML and recompiling.

C4. Setting up a trial — `reset_to_stand()` *(lines 104–113)*

Puts every joint at its `STAND_POS` angle **and** sets `data.ctrl` to the same value. So the robot starts exactly where it is being asked to be — zero error, zero torque at t = 0. Everything after that is gravity acting and the controller responding.

C5. Measuring lean — `tilt_deg()` *(lines 115–119)*

Takes the pelvis's local "up" axis and measures its angle from world up. 0° is perfectly upright, 90° is horizontal. This is the fall detector.

C6. One trial — `trial()` *(lines 121–188)*

The main loop, once per `dt`:

1. **optional shove** — at t = 1 s, add `perturb` to the base's sideways velocity
2. **step the physics** — `mujoco.mj_step()` advances 2 ms
3. **check for blow-up** — `data.warning[mjWARN_BADQACC].number > 0`
4. **check for a fall** — pelvis below 60% of start height, **or** tilt above 45°
5. **record** — during the last second only, log joint errors and speeds
6. **track peak torque** — the largest `actuator_force` seen at any moment

> **Why the blow-up check reads a warning counter instead of testing for NaN:** when the solver explodes, MuJoCo detects it and silently *resets the state*. A few steps later the numbers look plausible again, even though velocities passed 10^6 in between. Checking `qpos` for NaN misses it entirely. The warning counter is the honest signal.

Returns a dictionary — those are the fields printed in Step 8.

C7. The sweep — `main()` *(lines 190–240)*

```
kps = [25, 50, 100, 200, 400, 600, 900]
kds = [0, 1, 5, 15, 30, 60, 120]
```

Two modes:

- `--kp X --kd Y` given → one trial, print every metric
- **neither given** → all 7 × 7 = 49 combinations, print a grid, write `exp1_pd_sweep.csv`

Both lists are roughly logarithmic. That is deliberate: the interesting behaviour spans two orders of magnitude, and evenly spaced values would waste most trials in regions where nothing changes.

PART D — What you are allowed to change

D1. From the command line — no code editing

Flag	Default	Try	What happens
`--kp`	*(sweep)*	25 → 900	the spring. Below ~400 it topples
`--kd`	*(sweep)*	0 → 120	the syrup. Above ~30 the simulator blows up
`--seconds`	10	3 → 30	trial length. Shorter = faster sweep, same island
`--perturb`	0	0.05 → 0.4	sideways shove at t = 1 s, in m/s
`--out`	`exp1_pd_sweep.csv`	any path	where results are written
`--xml`	auto	a path	use a different robot model

D2. In the code — small edits, big questions

What	Where	Try	The question it answers
`STAND_POS` knee value	line ~81	0.3 → 0.6	does a deeper crouch widen the stable island?
`STAND_POS` ankle pitch	line ~81	−0.2 → −0.3	can you buy stability by leaning back?
`kps` / `kds` lists	line ~210	add 1200, or 45	where exactly is the edge?
`fall_frac`	line 121	0.6 → 0.8	a stricter definition of "fell" — does the map change?
tilt threshold `45.0`	line ~163	45 → 30	same question from the other direction
`model.opt.timestep`	set after `build()`	0.002 → 0.001	**the good one.** Halve the timestep and watch the blow-up region shrink — proving it was never the robot

That last row is the best experiment in this table. It demonstrates, in one run, that a whole class of "failures" belonged to the tool rather than the machine.

D3. What NOT to change

Thing	Why
The three lines in `set_gains()`	that is the experiment
The `mjWARN_BADQACC` check	you will start recording blow-ups as successes
`reset_to_stand()`	trials stop being comparable
The XML mesh references	the model will not load

Check yourself

1. You set one `kp`. Why does each joint produce a different torque?
2. A joint holds 9.6 N·m at `kp = 200`. What is its permanent error, in degrees?
3. Why does the *lightest* joint decide the damping limit for the whole robot?
4. The robot falls at `kp = 100`, using 17 N·m — when 31 N·m was available. Why?
5. Which of the two failure modes would still exist on a real robot?

(Answers: 1 — different error and speed at each joint, A3. 2 — 9.6/200 = 0.048 rad ≈ 2.8°, A5. 3 — the stability bound scales with inertia, and one runaway joint drags the chain, A6. 4 — it is leaning, not weak; a balance failure, A5. 5 — only the toppling. The blow-up is numerical, A6.)

Appendix B — Instructor Notes

- **This manual replaces the bundled version.** Nothing is distributed but the manual itself: no `r1_lab1_files.zip`, no prepared folder. Students install the tools, clone the model from Unitree's public repository, and create all eight files by pasting from the manual. Ship the `.md` alongside the PDF — copying Python out of a PDF loses indentation, and the `.md` is the copy source.
- **Part 0 is the only part that can fail unrecoverably**, and WSL2 is the riskiest single step (virtualization disabled in BIOS, corporate images, no reboot rights). Send Part 0 a week early and require the four version numbers from 0g before the session.
- **Parts 1–4 are new and are the point of the redesign.** A student who can rebuild the folder in Part 11 has learned something no zip file could teach.
- **Part 3's table is the intellectual core of the first hour.** The actuators — the subject of the whole experiment — do not exist in the file Unitree publishes. Do not let that pass unremarked.
- **Part 6 is the lab.** If they leave able to separate a physical failure from a numerical artifact, it worked — even if `kp` never fully lands.
- **Do not reveal the map before Part 7.** The prediction is what makes the reveal land.
- **Insist on `kd = 5` for push tests.** Students using 30 will report the simulator's limit.
- **The live viewer cannot be relied on.** `gladLoadGL error` reproduced under WSL2 on the author's machine in every configuration tried, GPU included; offscreen rendering was unaffected. Part 9a is the path that always works — plan the room around it and treat 9b/9c as a bonus.
- **Timing:** Part 0 about 40 minutes at home. Parts 1–4 about 40 minutes, mostly pasting. Parts 5–8 about 35 including the 6-minute sweep. Parts 9–11 open-ended.
- **Verified end to end on 2026-09-01** on Ubuntu 24.04 under WSL2 — fresh clone, fresh workspace, every command in this manual, MuJoCo 3.12.0 / numpy 2.4.6 / matplotlib 3.11.1. Every number in every `Expected` block was captured from that run. macOS remains unexercised.